

SCOPE OF APPLICATION All Project/Engineering		SHT/SHTS 1 / 60
Responsibility: 제어SW플랫폼1팀	AUTOSAR CryptoLib User Manual	DOC. NO
AUTOSAR CryptoLib User Manual		

Document Change History				
Date (YYYY-MM-DD)	Ver.	Editor	Chap	내용(개정 전 -> 개정 후)
2018-02-09	1.0	강신일	전체	• 신규제정
2018-02-09	1.1	강신일	2.3	• Release 정보 갱신
2018-03-19	1.2	강신일	2.3	• Release 정보 갱신
2018-04-20	1.3	강신일	2.3 4.1	• Release 정보 갱신 • Hsm_DataType, Hsm_DataIDType 수정
2018-04-30	1.4	강신일	2.3	• Release 정보 갱신
2018-05-11	1.5	강신일	2.3 3.3	• Release 정보 갱신 • CryptoLibAutronIcus 설정 추가
2018-09-04	1.6	강신일	2.3	• Release 정보 갱신
2018-10-25	1.7	김은경	2.3	• Release 정보 갱신
2018-11-16	1.8	강신일	2.3	• Release 정보 갱신
2018-12-12	1.9	강신일	2.3	• Release 정보 갱신
2018-12-21	1.10	강신일	2.3	• Release 정보 갱신
2019-04-24	1.11	강신일	2.3 3.1 4, 5	• Release 정보 갱신 • 누락 파라미터 정보 추가 • RH850 F1K ICUS 설명 추가
2019-06-13	1.12	강신일	2.3 4.2.2.17	• Release 정보 갱신 • Icus_VerifyProtectedKey 인자 변경
2019-07-05	1.13	강신일	2.3 3.4, 4.1.3, 4.2.3, 5.1.3	• Release 정보 갱신 • S32K CSEC 설명 추가
2019-10-17	1.14	강신일	2.3	• Release 정보 갱신 • 파일 구조 변경 (lib header 파일을 delivery로 이동)
2019-12-05	1.15	강신일	2.3 3.1	• Release 정보 갱신 • Support Hsm 리스트 변경
2019-12-23	1.16	강신일	2.3	• Release 정보 갱신
2020-03-13	1.17	임재현	2.4	• Release 정보 갱신

17th Edition Date : 2019-12-23	File Name: AUTRON_AUTOSAR_CryptoLib_Modul e_UM.pdf	Creation 강신일	Check 윤영진	Approval 백종현
Document Management System		2019-12-23	2019-12-23	2019-12-23

2020-07-01

3.4.0.0

임재현

2.4

• Release 정보 갱신

목 차

1. OVERVIEW	4
2. PRODUCT RELEASE NOTES	5
2.1 OVERVIEW	5
2.2 SCOPE OF THE RELEASE	5
2.3 CHANGE LOG	5
2.3.1 Version 2.2.0	5
2.3.2 Version 2.2.1	5
2.3.3 Version 2.2.2	5
2.3.4 Version 2.3.0	6
2.3.5 Version 2.3.1	7
2.3.6 Version 2.3.2	7
2.3.7 Version 2.4.0	7
2.3.8 Version 2.5.0	8
2.3.9 Version 2.5.1	8
2.3.10 Version 2.5.2	8
2.3.11 Version 3.0.0	8
2.3.12 Version 3.1.0	8
2.3.13 Version 3.1.1	9
2.3.14 Version 3.2.0	9
2.3.15 Version 3.2.0.0	9
2.3.16 Version 3.3.0.0	10
2.3.17 Version 3.3.1	10
2.3.18 Version 3.4.0.0	10
2.4 LIMITATIONS	10
2.5 DEVIATIONS	11
3. CONFIGURATION GUIDE	12
3.1 CRYPTOLibGENERAL CONTAINER	12
3.2 CRYPTOLibAUTRONHSM CONTAINER	12
3.3 CRYPTOLibAUTRONICUS CONTAINER	12
3.4 CRYPTOLibAUTRONCSEC CONTAINER	12
3.5 CRYPTOLibHAEHSM CONTAINER	14
4. APPLICATION PROGRAMMING INTERFACE (API)	15
4.1 TYPE DEFINITIONS	15
4.1.1 Autron HSM 프레임워크	15
4.1.2 RH850 F1K 계열 ICUS	16
4.1.3 S32K 계열 CSEC	23
4.2 FUNCTIONS	27
4.2.1 Autron HSM 프레임워크	27
4.2.2 RH850 F1K 계열 ICUS	29
4.2.3 S32K 계열 CSEC	39
5. APPENDIX	49
5.1 API 사용시 유의사항	49
5.1.1 Autron HSM 프레임워크	49
5.1.2 RH850 F1K 계열 ICUS	55

5.1.3	S32K 계열 CSEC	59
-------	--------------------	----

1. Overview

CryptoLib 은 HSM 드라이버를 제공하며, 사용자는 제공된 HSM 드라이버 API 를 통해 HSM 제공 기능을 사용할 수 있다.

설정관련 Category의 해석은 다음과 같다.

- Changeable (C) : User 에 의해서 설정 가능한 항목
- Fixed (F) : User 에 의한 변경이 불가능한 항목
- NotSupported (N) : 사용되지 않는 항목

2. Product Release Notes

2.1 Overview

이 Chapter에서는, 현대오트론 CryptoLib Module에 대한 release 관련 내용을 제공하는데 목적이 있으며, CryptoLib Software product release version 에 대한, 제한사항 및 특이사항을 기술하고 있다.

2.2 Scope of the release

이 문서에 대한 모든 내용은, 다음의 현대오트론 CryptoLib 모듈에 한정한다.

Module	Autosar version	Module version
CryptoLib	4.0.3 이후	3.4.0.0

2.3 Change Log

2.3.1 Version 2.2.0

➤ 신규 기능

■ CryptoLib Generator 구현

원인	CryptoLib Generator 구현. Mcu 별 적용 HSM 드라이버 설정
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.2 Version 2.2.1

➤ 신규 기능

■ MAC generation / verification 처리 분리

원인	Chorus HSM 의 MAC service 내부 처리를 분리함
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.3 Version 2.2.2

➤ 신규 기능

■ SPC58x HSM 코어 구동 SW 코드 제거

원인	SPC58x HSM 코어는 HSM Enable 적용 시 Boot 직후 자동 시작된다. 따라서 불필요한 Host 코어에서 SW 로 HSM 코어 구동 코드를 제거한다.
----	---

동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.4 Version 2.3.0

➤ 신규 기능

■ Default ISK 추출 기능 추가

원인	<IAU 제어기 특화 기능> 선주입된 Default ISK 추출 기능 추가
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 개선 사항

■ IAU 제어기 특화 주입 ECC 키 쌍 PSK 적용

원인	IAU 제어기 특화 주입한 ECC 키 쌍을 암호화 알고리즘의 키입력시 PSK 와 같이 인덱스로 사용 가능하도록 개선
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

■ SPC58x HAC Random 라이브러리 추가

원인	SPC58x HAC Random 라이브러리 추가
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

■ IAU 특화 데이터 Default ISK 추가

원인	IAU 특화 데이터 Default ISK 추출 기능 추가
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

■ HAE HSM 드라이버 업데이트

원인	Aurix TC27x HSM 드라이버 라이브러리 & 헤더 파일 업데이트
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.5 Version 2.3.1

➤ 개선 사항

■ SPC58x HSM 드라이버 FBL reprogramming 지원

원인	FBL reprogramming 시 SCP58x HSM 동작 제어 API 제공
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.6 Version 2.3.2

➤ 신규 기능

■ SPC58x HSM 용 ECDH 알고리즘 드라이버 추가

원인	SPC58x HSM 용 ECDH 알고리즘 드라이버 추가 구현
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 개선 사항

■ ICUS 내부 timeout 값 설정 기능 구현

원인	ICUS 사용시 ICUS 내부 응답의 timeout 체크를 위한 값 설정 파라미터화
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.7 Version 2.4.0

➤ 신규 기능

■ RH850 F1KM HSM 지원

원인	RH850 F1KM HSM 지원 드라이버 구현
동작 영향	없음
설정 영향	없음

ASW 조치 필요 사항	없음
--------------	----

2.3.8 Version 2.5.0

➤ 신규 기능

■ Aurix TC29x HAE HSM 드라이버 업데이트

원인	Aurix TC29x HSM 드라이버 라이브러리 & 헤더 파일 업데이트
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.9 Version 2.5.1

➤ 신규 기능

■ HAE 암호화 알고리즘 라이브러리 추가

원인	SPC58X, MPC560XB 용 HAE 암호화 알고리즘 SW 라이브러리 추가
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.10 Version 2.5.2

➤ 개선 사항

■ Chorus 2M/4M/6M 지원

원인	SPC58x 계열 설정 분리
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.11 Version 3.0.0

➤ 개선 사항

■ 암호화 라이브러리 파일의 모듈 외 분리

원인	기존에 모듈 내에 포함되어 있던 암호화 라이브러리를 모듈 외부로 분리
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	암호화 라이브러리 배포 프로세스에 따른 배포 요청을 통해 받은 라이브러리를 직접 통합, 사용해야 함

2.3.12 Version 3.1.0

➤ 개선 사항

■ ICUS 드라이버 제공

원인	기존에 플랫폼 내부에서만 사용한 ICUS API 의 사용자 직접 사용 지원
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	API 사용 가이드 참조

■ ICUS CMD_DEBUG 지원 여부 설정화

원인	ICUS 의 NVM Key 를 삭제하고자 할 경우 CMD_DEBUG command 가 지원되어야 한다.
동작 영향	없음
설정 영향	CryptoLib / CryptoLibAutronIcus / Support Debug Command 설정 (SRS 요청 필요)
ASW 조치 필요 사항	CMD_DEBUG command 를 지원하기 위한 Icus_Debug API 내부에서 third-party 알고리즘을 사용하므로 해당 command 사용 여부에 따라 HAE CryptoLib 라이브러리의 통합이 필요함

2.3.13 Version 3.1.1

➢ 개선 사항

■ ICUS key 등록시 key flag 처리 오류 수정

원인	ICUS Key flag 의 하위 2 bit (DISABLE_WILDCARD, CMAC_USAGE) 미적용 오류 수정
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	Icus_VerifyProtectedKey 사용시 미사용 key flag 인자 제거

2.3.14 Version 3.2.0

➢ 개선 사항

■ CSEC 드라이버 제공

원인	CSEC API 의 사용자 직접 사용 지원
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	API 사용 가이드 참조

2.3.15 Version 3.2.0.0

➢ 개선 사항

■ 파일 구조 변경

원인	CryptoLib 과 연계된 Csm 모듈의 Code Open 을 위해 파일 구조 변경
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.16 Version 3.3.0.0

➤ 개선 사항

■ 오토에버 HSM 2.0 지원

원인	오토에버 HSM 2.0 인터페이스 지원
동작 영향	오토에버 HSM 1.X 미지원
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.17 Version 3.3.1

➤ 개선 사항

■ 모듈 버전 변경

원인	소스 코드 미공개 모듈로 버전 형식 변경
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.3.18 Version 3.4.0.0

➤ 개선 사항

■ Autoever HSM 관련 함수 라이브러리 영향성 제거

원인	Autoever HSM 사용에 있어서 library 의존성이 존재함. Autron HSM 을 제외한 Function 에 대해서는 Library 를 제외함.
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.4 Limitations

오토에버 HSM 1.X 미지원

2.5 Deviations

N/A

3. Configuration Guide

3.1 CryptoLibGeneral Container

Parameter Name	Value	Category
Short Name	User Defined	C
Support Hsm ¹⁾	HAE_HSM_2_X / ICUS / ICUM / SPC584BX / SPC584CX / SPC58ECX / SPC58XEX / SPC58XGX	F
Enable Dev Info ²⁾	true / false	F

- 1) 플랫폼에서 지원하는 HSM 의 MCU 설정
- 2) HSM 드라이버 동작 관련 디버깅 정보 on/off

3.2 CryptoLibAutronHsm Container

Parameter Name	Value	Category
Short Name	User Defined	C
Use Os ¹⁾	true / false	F
SupportECU ²⁾	IAU	F

- 1) Autron HSM 드라이버 내의 Os 사용 유무 선택. false 선택시 sync 동작만 지원
- 2) Autron HSM 드라이버 & 프레임워크가 특화 기능 지원하는 제어기

3.3 CryptoLibAutronIcus Container

Parameter Name	Value	Category
Short Name	User Defined	C
TimeoutLevel1 ¹⁾	0 ~ 4294967295	F
TimeoutLevel2 ¹⁾	0 ~ 4294967295	F
TimeoutLevel3 ¹⁾	0 ~ 4294967295	F
TimeoutLevel4 ¹⁾	0 ~ 4294967295	F
Support Debug Command ²⁾	true / false	F

- 1) ICUS 응답 timeout 체크 기준값
- 2) ICUS Non-volatile memory Key, RAM Key 삭제를 위한 CMD_DEBUG command 의 사용 여부. true 설정시 3rd party 암호화 알고리즘 라이브러리 (HAE CryptoLib) 이 필요

3.4 CryptoLibAutronCsec Container

Parameter Name	Value	Category
Short Name	User Defined	C
Timeout ¹⁾	0 ~ 4294967295	C
Flash Key Size ²⁾	CSEC_NO_FLASH_KEYS / CSEC_5_FLASH_KEYS / CSEC_10_FLASH_KEYS / CSEC_20_FLASH_KEYS	C
Security Flag Extension ³⁾	CSEC_FLASH_KEY_VERIFY_ONLY_DISABLED / CSEC_FLASH_KEY_VERIFY_ONLY_ENABLED	C
FlexRAMLoadOption ⁴⁾	CSEC_FLEXRAM_LOADED_EEPROM_DATA / CSEC_FLEXRAM_NOT_LOADED	C
EEPROMSize ⁵⁾	CSEC_EEPROM_0B / CSEC_EEPROM_2KB / CSEC_EEPROM_4KB	C
EEPROMBackupSize ⁶⁾	CSEC_EEPROM_BACKUP_0B / CSEC_EEPROM_BACKUP_24KB / CSEC_EEPROM_BACKUP_32KB / CSEC_EEPROM_BACKUP_48KB / CSEC_EEPROM_BACKUP_64KB	C
Secure Boot Size ⁷⁾	0 ~ 4294967295	C
Secure Boot Flavor ⁸⁾	CSEC_SECURE_BOOT_NONE / CSEC_SECURE_BOOT_SERIAL / CSEC_SECURE_BOOT_PARALLEL / CSEC_SECURE_BOOT_STRICT	C
Secure Boot Apply ⁹⁾	true / false	C

- 1) CSEC 응답 timeout 체크 기준값
- 2) CSEC Non-volatile memory Key 사용 개수. FLASH_KEY 사용시 MASATER_ECU_KEY, BOOT_MAC_KEY, BOOT_MAC 를 포함한 개수이므로 실제 사용할 수 있는 KEY_<n> 의 개수가 3 개 줄어듦.
ex) CSEC_20_FLASH_KEYS 설정시 KEY_1 ~ KEY_17 사용 가능
 - CSEC_NO_FLASH_KEYS : CSEC 미사용
 - CSEC_5_FLASH_KEYS : 1 to 5 keys
 - CSEC_10_FLASH_KEYS : 1 to 10 keys
 - CSEC_20_FLASH_KEYS : 1 to 20 keys
- 3) Non-volatile memory Key 등록시 Key attribute 중 VERIFY_ONLY 속성을 적용 선택 옵션
 - CSEC_FLASH_KEY_VERIFY_ONLY_DISABLED : VERIFY_ONLY 미적용
 - CSEC_FLASH_KEY_VERIFY_ONLY_ENABLED : VERIFY_ONLY 적용
- 4) Reset 과정중 valid EEPROM data 를 FlexRAM 에 load 적용 선택 옵션
 - CSEC_FLEXRAM_LOADED_EEPROM_DATA : CSEC 사용시 반드시 적용
 - CSEC_FLEXRAM_NOT_LOADED : 기능 미적용
- 5) EEPROM emulation 을 위한 FlexRAM 크기
 - CSEC_EEPROM_0B : 기능 미적용

- CSEC_EEPROM_2KB / CSEC_EEPROM_4KB : CSEC 사용시 반드시 적용
 - S32K116 / S32K118 : CSEC_EEPROM_2KB
 - S32K142 / S32K144 / S32K146 / S32K148 : CSEC_EEPROM_4KB
- 6) EEPROM emulation backup 을 위한 Data flash 할당 크기
- CSEC_EEPROM_BACKUP_0B : 기능 미적용
 - CSEC_EEPROM_BACKUP_24KB/32KB/48KB/64KB : CSEC 사용시 반드시 적용. 적용 가능 크기는 MCU 매뉴얼 참조
- 7) PFlash '0' 주소부터 Secure Boot 를 적용할 **bit** 크기. 최대 512KB
- 8) Secure Boot 설정 적용 방식 옵션.
- Secure Boot 실패 여부 및 개별 Non-volatile memory key 속성 설정에 따라 key 사용 가능 여부 결정됨.
- CSEC_SECURE_BOOT_NONE : Secure Boot 기능 비활성화
 - CSEC_SECURE_BOOT_SERIAL : Secure Boot 수행 이후 Main core 시작
 - CSEC_SECURE_BOOT_PARALLEL : Main core 시작과 병렬로 Secure Boot 수행
 - CSEC_SECURE_BOOT_STRICT : CSEC_SECURE_BOOT_SERIAL 방식과 같으나 **설정 적용 후 취소 불가**
- 9) Secure Boot 설정 적용 여부
- Reset 후 플랫폼 모듈 초기화 과정에서 Secure Boot 설정 적용 로직 동작

3.5 CryptoLibHaeHsm Container

Parameter Name	Value	Category
Short Name	User Defined	C
Timeout ¹⁾	0 ~ 4294967295	C
Max Error Count ²⁾	0 ~ 4294967295	C

- 1) HSM_WaitResp 응답 timeout 체크 기준값
- 2) HSM_GetLastErrorCode 로그 최대 개수

4. Application Programming Interface (API)

오토에버 HSM 적용 플랫폼의 경우 오토에버 HSM User Manual 을 참조.

4.1 Type Definitions

4.1.1 Autron HSM 프레임워크

4.1.1.1 Hsm_DataType

Name	Hsm_DataType	
Type	Enumeration	
Range	HSM_DATA_INVALID_KEY	
	HSM_DATA_PSK_AES_KEY	PSK(추출 불가)
	HSM_DATA_PSK_RSA_PUBLIC_KEY	
	HSM_DATA_PSK_RSA_PRIVATE_KEY	
	HSM_DATA_PSK_ECC_PUBLIC_KEY	
	HSM_DATA_PSK_ECC_PRIVATE_KEY	
	HSM_DATA_AES_RAM_KEY	RAM KEY(추출 불가)
	HSM_DATA_RSA_RAM_PUBLIC_KEY	
	HSM_DATA_RSA_RAM_PRIVATE_KEY	
	HSM_DATA_ECC_RAM_PUBLIC_KEY	
	HSM_DATA_ECC_RAM_PRIVATE_KEY	
	HSM_DATA_CUSTOM_IAU_PUBLIC_KEY	IAU 제어기 전용 데이터(추출 가능)
	HSM_DATA_CUSTOM_IAU_PRIVATE_KEY	IAU 제어기 전용 데이터(추출 불가)
	HSM_DATA_CUSTOM_IAU_CERT	IAU 제어기 전용 데이터(추출 가능)
	HSM_DATA_CUSTOM_IAU_UID	
	HSM_DATA_CUSTOM_IAU_DEFAULT_ISK	
Description	Hsm 내부 저장 데이터 포맷 정보	

4.1.1.2 Hsm_DataIDType

Name	Hsm_DataIDType		
Type	uint32		
Range	Hsm_DataType	Value	
	HSM_DATA_PSK_AES_KEY	0~89	제어기 용도 사용불가 (HKMC 전용)
		90~99	제어기 용도 사용
	HSM_DATA_PSK_RSA_PUBLIC_KEY	0~49	제어기 용도 사용
	HSM_DATA_PSK_RSA_PRIVATE_KEY	0~49	제어기 용도 사용
	HSM_DATA_PSK_ECC_PUBLIC_KEY	0~49	제어기 용도 사용
	HSM_DATA_PSK_ECC_PRIVATE_KEY	0~49	제어기 용도 사용
	HSM_DATA_CUSTOM_IAU_KEY	100	IAU 제어기 전용 사용
Description	Hsm 내부 저장 데이터 고유 ID 사용하고자 하는 Hsm_DataType 에 따라 Index 의 범위가 다르다. 허용 범위 내의 가능한 key index 를 사용한다.		

4.1.1.3 Hsm_CallbackPtr

Name:	Hsm_CallbackPtr
Type	Function pointer
Description:	비동기 동작 완료시 호출되는 함수 포인터 void (*Hsm_CallbackPtr) (Hsm_ErrType ddRet)
Name	Hsm_CallbackPtr
Type	Function pointer
Description	비동기 동작 완료시 호출되는 함수 포인터 void (*Hsm_CallbackPtr) (Hsm_ErrType ddRet)

4.1.1.4 Hsm_ErrType

Name	Hsm_ErrType	
Type	Enumeration	
	HSM_NO_ERROR	정상
	HSM_BUSY	이전 요청 처리중
	HSM_NO_INITIALIZED_ERROR	프레임워크 초기화 실패
	HSM_RNG_NO_INITIALIZED_ERROR	PRNG 초기화 실패
	HSM_SHARED_MEM_ERROR	프레임워크 공유 메모리 초기화 실패
	HSM_INVALID_COMMAND	Command 오류
	HSM_PARAMETER_ERROR	Command 인자 오류
	HSM_CRYPTO_EXEC_ERROR	암호화 알고리즘 오류
	HSM_RNG_RESEED_ERROR	PRNG Seed 오류. 재시도 필요
	HSM_TIMEOUT_ERROR	암호화 알고리즘 timeout
	HSM_MEMORY_FAILURE	In/out 버퍼 오류
	HSM_FLASH_ERROR	Flash read/write 오류
	HSM_KEY_INVALID_ERROR	Data(Key) Index 오류
	HSM_KEY_EMPTY_ERROR	Ram Key Empty
	HSM_SECURE_BOOT_FAILED	Secure Boot 실패
HSM_NO_DEBUGGING	Secure Debug disabled	
Description	Hsm 에러 정보	

4.1.1.5 Hsm_SecureBootStatusType

Name	Hsm_SecureBootStatusType	
Type	Enumeration	
	HSM_SECURE_BOOT_NOT_TESTED	비활성화
	HSM_SECURE_BOOT_COMPARE_SUCCESS	Boot MAC 무결성 검증 성공
	HSM_SECURE_BOOT_COMPARE_FAILED	Boot MAC 무결성 검증 실패(위변조 발생)
	HSM_SECURE_BOOT_NO_BOOT_MAC_KEY_FAILED	Boot MAC Key 부재
	HSM_SECURE_BOOT_KEY_LOAD_FAILED	Boot MAC Key Flash read 오류
	HSM_SECURE_BOOT_MAC_LOAD_FAILED	Boot MAC Flash write 오류
	HSM_SECURE_BOOT_MAC_COMPUTE_FAILED	Boot MAC 계산 오류
Description	Secure Boot 상태 정보	

4.1.2 RH850 F1K 계열 ICUS

4.1.2.1 Icus_CmdType

Name	Icus_CmdType
------	--------------

Type	Enumeration	
	CMD_ENC_ECB	This command encrypts an integer multiple (n) of 128 bits of plain text in ECB mode of AES
	CMD_ENC_CBC	This command encrypts an integer multiple (n) of 128 bits of plain text in CBC mode of AES
	CMD_DEC_ECB	This command decrypts an integer multiple (n) of 128 bits of cipher text in ECB mode of AES
	CMD_DEC_CBC	This command decrypts an integer multiple (n) of 128 bits of cipher text in CBC mode of AES
	CMD_GENERATE_MAC	This command generates MAC of the message with the specified bit length of the message
	CMD_VERIFY_MAC	This command generates MAC of the message with the specified bit length of the message and verifies the MAC for the number of the bits specified MAC length
	CMD_LOAD_KEY	This command updates a key
	CMD_LOAD_PLAIN_KEY	This command registers the plain text key in the RAM_KEY memory slot
	CMD_EXPORT_RAM_KEY	This command exports the data in the RAM_KEY memory slot in a secure format. This command can only output RAM_KEY values that have been registered by CMD_LOAD_PLAIN_KEY
	CMD_INIT_RNG	This command generates a seed of a random number from the TRNG
	CMD_EXTEND_SEED	This command extends the seed of the pseudo-random number by 128 bits of entropy
	CMD_RND	This command outputs a 128-bit random number. At the same time, it updates the seed of the pseudo-random number
	CMD_SECURE_BOOT	When BOOT_MAC_KEY and BOOT_MAC are registered, this command checks if the MAC of the data provided as input matches the BOOT_MAC value. If BOOT_MAC slot is empty, a generated BOOT_MAC value is registered in the BOOT_MAC slot
	CMD_BOOT_FAILUERE	This command sets the BOOT_FINISHED flag in the ICUS status register, and clears the BOOT_OK flag
	CMD_BOOT_OK	This command sets the BOOT_FINISHED flag in the ICUS status register
	CMD_GET_ID	This command calculates the MAC of CHALLENGE, UID and SREG by using MASTER_ECU_KEY on the ICUS side based on the CHALLENGE value conveyed from the CPU and returns it with a 120-bit UID and 8-bit SREG
	CMD_DEBUG	This comamnd verifies the authentication MAC value (AUTHORIZATION) against a 128-bit random number (CHALLENGE). If the MAC value is correct, the INT_DEBUGGER flag in the ICUS status register is set and all the keys registered in RAM_KEY and Non-volatile memory are cleared. It also clears the RND_INIT flag in the ICUS status register
	CMD_CANCEL	This command cancels command processing and clears the BUSY flag in the ICUS status register
Description	Icus command 정보. SHE 사양의 User-accessible Functions 의 command 리스트와 1:1 매칭되며 드라이버 내의 Icus_<Command> 함수 사용이 가능하다.	

4.1.2.2 Icus_ErrType

Name	Icus_ErrType	
Type	Enumeration	
	ERC_NO_ERROR	Normal end
	ERC_SEQUENCE_ERROR	* CMD_SECURE_BOOT - After execution of CMD_SECURE_BOOT, the command is re-executed - After return from STOP mode, CMD_SECURE_BOOT was executed OP_NSBF_PSM in OPBT9 was 1 - After return from STOP mode, CMD_SECURE_BOOT was re-executed OP_BCLR_PSM in OPBT9 was 0
	ERC_KEY_NOT_AVAILABLE	* CMD_ENC_ECB / CMD_DEC_ECB, CMD_ENC_CBC / CMD_DEC_CBC, CMD_GENERATE_MAC / CMD_VERIFY_MAC - The BOOT_PROTECTION flag of the key to be used was 1 while CMD_SECURE_BOOT was not done - CMD_SECURE_BOOT was executed while BOOT_FINISHED and BOOT_OK in the ICUS status register were 1 and 0 respectively, and when the BOOT_PROTECTION flag of the key to be used was 1 - The DEBUGGER_PROTECTION flag of the key to be used was 1 while EXT_DEBUGGER in the ICUS status register was 1 * CMD_LOAD_KEY - When RAM_KEY was registerd, while CMD_SECURE_BOOT was not executed, and when the BOOT_PROTECTION flag of the AuthID key was 1 - When RAM_KEY was registerd, while BOOT_FINISHED and BOOT_OK in the ICUS status register were 1 and 0 respectively, and when the BOOT_PROTECTION flag of the AuthID key was 1 - When RAM_KEY is registered, the DEBUGGER_PROTECTION flag of the AuthID key was 1 while EXT_DEBUGGER in the ICUS status register was 1 * CMD_EXPORT_RAM_KEY - The BOOT_PROTECTION flag of MASTER_ECU_KEY was 1 while CMD_SECURE_BOOT was not done - CMD_SECURE_BOOT was executed while BOOT_FINISHED and BOOT_OK in the ICUS status register were 1 and 0 respectively, and when the BOOT_PROTECTION flag of the key to be used was 1 - The DEBUGGER_PROTECTION flag of MASTER_ECU_KEY was 1 while EXT_DEBUGGER in the ICUS status register was 1 * CMD_SECURE_BOOT - The DEBUGGER_PROTECTION flag in BOOT_MAC_KEY or BOOT_MAC was 1 while EXT_DEBUGGER in the ICUS status register was 1
	ERC_KEY_INVALID	* CMD_ENC_ECB / CMD_DEC_ECB, CMD_ENC_CBC / CMD_DEC_CBC - The value specified by KEYID has not been defined - A key other than KEY_<n> or RAM_KEY was specified - A key where KEY_USAGE of KEY_FLAG is 1 was specified CMD_GENERATE_MAC - The value specified by KEYID has not been defined

		- A key other than KEY_<n> or RAM_KEY was specified - A key where KEY_USAGE of KEY_FLAG is 0 was specified - A key where CMAC_USAGE of KEY_FLAG is 1 was specified * CMD_VERIFY_MAC - The value specified by KEYID has not been defined - A key other than KEY_<n>, RAM_KEY or BOOT_MAC_KEY was specified - A key where KEY_USAGE of KEY_FLAG is 0 was specified * CMD_LOAD_KEY - A problem occurred in specifying the AuthID value for the ID value when M1 was specified * CMD_EXPORT_RAM_KEY - The registered RAM_KEY was registered by the CMD_LOAD_KEY command
	ERC_KEY_EMPTY	* CMD_ENC_ECB / CMD_DEC_ECB, CMD_ENC_CBC / CMD_DEC_CBC, CMD_GENERATE_MAC - The specified KEY_<n> or RAM_KEY is empty CMD_VERIFY_MAC - The specified KEY_<n>, RAM_KEY or BOOT_MAC_KEY is empty * CMD_LOAD_KEY - The key indicated by AuthID value specified in M1 was empty, except when the ID value and the AuthID value are the same * CMD_EXPORT_RAM_KEY - RAM_KEY was empty * CMD_DEBUG - MASTER_ECU_KEY was empty
	ERC_NO_SECURE_BOOT	* CMD_SECURE_BOOT - BOOT_MAC_KEY was empty * CMD_BOOT_FAILURE, CMD_BOOT_OK - This command was executed while the ICUS status register did not satisfy the condition SECURE_BOOT = 1 & BOOT_OK = 1 & BOOT_FINISHED = 0
	ERC_KEY_WRITE_PROTECTED	* CMD_LOAD_KEY - A key has already been written to the memory slot to be updated, and the WRITE_PROTECTION flag of the key is 1 * CMD_DEBUG - Among all the keys that have been registered in Non-volatile memory, the WRITE_PROTECTION flag for any of them is 1
	ERC_KEY_UPDATE_ERROR	* CMD_LOAD_KEY - M3 did not match the result of MAC calculation for M1 and M2 - The specified UID value did not match the UID value of the device - The UID value in M1 is specified as a Wildcard UID, but the DISABLE_WILDCARD flag of the key to be updated is 1 - A key has already been written to the memory slot to be updated, but the Counter value of the key is greater than or equal to the Counter value specified in M2

		- Any value except 0 is set to the counter value or KEY_FLAG specified in M2 when RAM_KEY is registered
	ERC_RNG_SEED	* CMD_EXTEND_SEED, CMD_RND - RND_INIT in the ICUS status register is 0 * CMD_DEBUG - This command was executed while RND_INIT in the ICUS status register was 0
	ERC_NO_DEBUGGING	* CMD_DEBUG - The specified AUTHORIZATION value is not correct
	ERC_BUSY	* CMD_ENC_ECB / CMD_DEC_ECB, CMD_ENC_CBC / CMD_DEC_CBC, CMD_GENERATE_MAC / CMD_VERIFY_MAC, CMD_LOAD_KEY, CMD_LOAD_PLAIN_KEY, CMD_EXPORT_RAM_KEY, CMD_INT_RNG, CMD_EXTEND_SEED, CMD_RND, CMD_SECURE_BOOT, CMD_BOOT_FAILURE, CMD_BOOT_OK, CMD_GET_ID, CMD_DEBUG - The ICUS was processing a command when this command was set
	ERC_MEMORY_FAILURE	* CMD_ENC_ECB / CMD_DEC_ECB, CMD_ENC_CBC / CMD_DEC_CBC, CMD_GENERATE_MAC / CMD_VERIFY_MAC, CMD_LOAD_KEY, CMD_EXPORT_RAM_KEY, CMD_GET_ID - An ECC error occurred due to a read access to the data flash memory by ICUS - An ECC error notification occurred due to a read access to the internal RAM of the ICUS * CMD_LOAD_PLAIN_KEY, CMD_INT_RNG, CMD_EXTEND_SEED, CMD_RND - An ECC error notification occurred due to a read access to the internal RAM of the ICUS * CMD_SECURE_BOOT, CMD_DEBUG - An error occurred while data was being written to a memory slot - An ECC error occurred due to a read access to the data flash memory by ICUS - An ECC error notification occurred due to a read access to the internal RAM of the ICUS
	ERC_GENERAL_ERROR	* CMD_ENC_ECB / CMD_DEC_ECB - The ICUS data transfer number register was 0 or 1 when this command was set * CMD_VERIFY_MAC - OP_MIN_ML_NV in OPBT9 is 0, and except under the following condition The number of bits of MAC's length >= the number of bits specified as the minimum MAC_LENGTH value by setting OPBT9 * CMD_INIT_RNG - When self-diagnosis test fails while the AIS32EN bit of ICUS command register is set to 1 * CMD_SECURE_BOOT - The specified value of SIZE was 0 byte
	ERC_TIMEOUT_ERROR	Internal timeout error
Description	Icus 에러 정보. ERC_TIMEOUT_ERROR 를 제외한 나머지 에러는 SHE 사양의 Error Codes 의 command 리스트와 1:1 매칭된다.	

4.1.2.3 Icus_KeyIDType

Name	Icus_KeyIDType	
Type	Enumeration	
	SECRET_KEY	This key is unique the the device. This key information is not open to the user. This key is used to register and acquire RAM_KEY
	MASTER_ECU_KEY	This key is the property of the owner of the device is used to erase, register and update other keys to be stored in the device
	BOOT_MAC_KEY	This key is used for the secure boot function. Writing this key enables the secure boot function. This key can also be used by the MAC verification command (CMD_VERIFY_MAC)
	BOOT_MAC	BOOT_MAC is the MAC value of the boot code to be verified at the time of secure booting. Although it is not used as a key, its value is managed and handled in the same way as the Non-volatile memory keys. If BOOT_MAC has not been set at the initial startup of secure booting, the MAC for the given boot code memory block is calculated and stored in the memory slot as BOOT_MAC. After that, at the startup of secure booting, the MAC value calculated from the given boot code memory block is compared with the registered BOOT_MAC, and the result of comparison is indicated in the ICUS status register
	KEY_1 KEY_2 KEY_3 KEY_4 KEY_5 KEY_6 KEY_7 KEY_8 KEY_9 KEY_10 KEY_11 KEY_12 KEY_13 KEY_14 KEY_15 KEY_16 KEY_17 KEY_18 KEY_19 KEY_20	KEY_<n> : These keys are used to encrypt and decrypt data and to generate and verify the message authentication (MAC). The intended usage of these keys is specified by the KEY_USAGE flag and the CMAC_USAGE flag in KEY_FLAG for the given key. <n> represents any number from 1 to 20
	RAM_KEY	This key is used to encrypt and decrypt data and to generate and verify a MAC. This key does not have KEY_FLAG, so specifying the intended use of this key is not necessary
	INVALID_KEY	Wrong key index
Description	Icus Key ID 정보.	

4.1.2.4 Icus_KeyFlagType

Name	Icus_KeyFlagType
------	------------------

Type	Enumeration	
	ALL_ENABLED	All restrictions are not set
	CMAC_USAGE	(CU) If this flag is set, the key cannot be used to execute CMD_GENERATE_MAC command
	DISABLE_WILDCARD	(DWC) If this flag is set, key update by the wildcard UID is disabled
	KEY_USAGE	(KU) If this flag is set, the key cannot be used with the CMD_ENC_ECB / CMD_DEC_ECB and CMD_ENC_CBC / CMD_DEC_CBC commands, but can be used with the CMD_GENERATE_MAC / CMD_VERIFY_MAC commands. However, the latter also depends on the setting of the CMAC_USAGE flag
	DEBUGGER_PROTECTION	(DP) If this flag is set, the key cannot be used when the EXT_DEBUGGER bit in the ICUS status register is 1
	BOOT_PROTECTION	(BP) If this flag is set, the key cannot be used when CMD_SECURE_BOOT is not executed, or BOOT_OK and BOOT_FINISHED in the ICUS01CRSTS register are set to 0 and respectively
	WRITE_PROTECTION	(WP) If this flag is set, rewriting and erasing of the key are disabled. This flag cannot be cleared once it has been set
Description	Icus Key Flag 정보.	

4.1.2.5 Icus_StatusType

Name	Icus_StatusType	
Type	Enumeration	
	BUSY	While command execution is started
	SECURE_BOOT	Checking the validity of BOOT_MAC_KEY after CMD_SECURE_BOOT is executed
	BOOT_INIT	CMD_SECURE_BOOT is executed without BOOT_MAC and when registration of BOOT_MAC is completed
	BOOT_FINISHED	* SECURE_BOOT = 1, BOOT_FINISHED = 0, and BOOT_OK = 1 and when CMD_BOOT_OK or CMD_BOOT_FAILURE is executed. * CMD_SECURE_BOOT is executed and if its value does not match that of BOOT_MAC * CMD_SECURE_BOOT is executed without BOOT_MAC and if BOOT_MAC_KEY is successfully registered * CMD_SECURE_BOOT is executed and if ERC_KEY_NOT_AVAILABLE, ERC_MEMORY_FAILURE, or ERC_GENERAL_ERROR occurs during the execution of the command * CMD_CANCEL is issued during execution of CMD_SECURE_BOOT and processing of the command is cancelled
	BOOT_OK	Execution of CMD_SECURE_BOOT is successfully completed and comparison with BOOT_MAC registered
	RND_INIT	Execution of CMD_INIT_RNG is successfully completed
	EXT_DEBUGGER	A debug interface is used
	INT_DEBUGGER	Execution of CMD_DEBUG is successfully completed
Description	Icus 내부 상태 정보. Icus_GetStatus API 를 통해 ICUS 내부 상태 정보를 확인할 수 있다.	

4.1.3 S32K 계열 CSEC

4.1.3.1 Csec_CmdType

Name	Csec_CmdType	
Type	Enumeration	
	CSEC_CMD_ENC_ECB	AES-128 encryption in ECB mode
	CSEC_CMD_ENC_CBC	AES-128 encryption in CBC mode
	CSEC_CMD_DEC_ECB	AES-128 decryption in ECB mode
	CSEC_CMD_DEC_CBC	AES-128 decryption in CBC mode
	CSEC_CMD_GENERATE_MAC	AES-128 based CMAC generation
	CSEC_CMD_VERIFY_MAC	AES-128 based CMAC verification
	CSEC_CMD_LOAD_KEY	Internal key update
	CSEC_CMD_LOAD_PLAIN_KEY	RAM key update
	CSEC_CMD_EXPORT_RAM_KEY	RAM key export
	CSEC_CMD_INIT_RNG	PRNG initialization
	CSEC_CMD_EXTEND_SEED	PRNG seed entropy extension
	CSEC_CMD_RND	Random number generation
	CSEC_CMD_BOOT_FAILUERE	Impose sanction during invalid boot
	CSEC_CMD_BOOT_OK	Finish boot verification
	CSEC_CMD_GET_ID	Get UID
	CSEC_CMD_BOOT_DEFINE	Secure boot configuration
	CSEC_CMD_DBG_CHAL	Get debug challenge
	CSEC_CMD_DBG_AUTH	Debug authentication
	CSEC_CMD_MP_COMPRESS	Miyaguchi-Preneel compression
Description	Csec command 정보. SHE 사양의 User-accessible Functions 의 command 리스트와 대부분 1:1 매칭되며 드라이버 내의 Csec_<Command> 함수 사용이 가능하다.	

4.1.3.2 Csec_ErrType

Name	Csec_ErrType	
Type	Enumeration	
	CSEC_ERC_NO_ERROR	No error has occurred and the command will be executed
	CSEC_ERC_SEQUENCE_ERROR	The sequence of commands or subcommands is out of sequence
	CSEC_ERC_KEY_NOT_AVAILABLE	key has DBG Attached flag and debugger is active
	CSEC_ERC_KEY_INVALID	A function is called to perform an operation with a key that is not allowed for the given operation
	CSEC_ERC_KEY_EMPTY	Key slot is empty (not initialized)/not present or higher slot (not partitioned)
	CSEC_ERC_NO_SECURE_BOOT	N/A for FTFC - BOOT_DEFINE once configured, will automatically run secure boot
	CSEC_ERC_KEY_WRITE_PROTECTED	A key update is attempted on a write protected key slot or the debugger is started while a key is write-protected
	CSEC_ERC_KEY_UPDATE_ERROR	A key update did not succeed due to errors in verification of the messages
	CSEC_ERC_RNG_SEED	The PRNG seed has not yet been initialized. (CSESTAT[RIN] != 1)
	CSEC_ERC_NO_DEBUGGING	Internal debugging is not possible because the authentication did not succeed

	CSEC_ERC_MEMORY_FAILURE	General memory technology failure (multi-bit ECC error, common fault detection)
	CSEC_ERC_GENERAL_ERROR	Detected error that is not covered by the other error codes (Example: null pointer received, FuncID is not valid, message length invalid, for pointer method GENERATE_MAC is the message length would cause the sequence to cross a read partition boundary)
	CSEC_ERC_BUSY	Detected error that another command is in progress
Description	Csec 에러 정보. SHE 사양의 Error Codes 의 command 리스트와 1:1 매칭된다.	

4.1.3.3 Csec_KeyIDType

Name	Csec_KeyIDType	
Type	Enumeration	
	CSEC_SECRET_KEY	Unique secret key
	CSEC_MASTER_ECU_KEY	Used for updating other memory slots
	CSEC_BOOT_MAC_KEY	Used by the secure booting mechanism to verify the authenticity of the software
	CSEC_BOOT_MAC	Stores the MAC of the Bootloader for the secure booting mechanism
	CSEC_KEY_1	User key
	CSEC_KEY_2	
	CSEC_KEY_3	
	CSEC_KEY_4	
	CSEC_KEY_5	
	CSEC_KEY_6	
	CSEC_KEY_7	
	CSEC_KEY_8	
	CSEC_KEY_9	
	CSEC_KEY_10	
	CSEC_KEY_11	
	CSEC_KEY_12	
	CSEC_KEY_13	
	CSEC_KEY_14	
	CSEC_KEY_15	
	CSEC_KEY_16	
	CSEC_KEY_17	
	CSEC_RAM_KEY	A volatile key that can be used for arbitrary operations
	CSEC_INVALID_KEY	Wrong key index
Description	Csec Key ID 정보.	

4.1.3.4 Csec_BootFlavorType

Name	Csec_BootFlavorType	
Type	Enumeration	
	CSEC_BOOT_STRICT	Strict Sequential Boot Mode
	CSEC_BOOT_SERIAL	Serial Boot Mode
	CSEC_BOOT_PARALLEL	Parallel Boot Mode
	CSEC_BOOT_NOT_DEFINED	No boot defined
Description	Csec Secure Boot method	

4.1.3.5 Csec_KeyFlagType

Name	Csec_KeyFlagType	
Type	Enumeration	
	CSEC_ALL_ENABLED	All restrictions are not set
	CSEC_DISABLE_WILDCARD	If this flag is set, key update by the wildcard UID is disabled
	CSEC_KEY_USAGE	If this flag is set, the key cannot be used with the CMD_ENC_ECB / CMD_DEC_ECB and CMD_ENC_CBC / CMD_DEC_CBC commands, but can be used with the CMD_GENERATE_MAC / CMD_VERIFY_MAC commands. However, the latter also depends on the setting of the CMAC_USAGE flag
	CSEC_DEBUGGER_PROTECTION	If this flag is set, the key cannot be used when the EXT_DEBUGGER bit in the ICUS status register is 1
	CSEC_BOOT_PROTECTION	If this flag is set, the key cannot be used when CMD_SECURE_BOOT is not executed, or BOOT_OK and BOOT_FINISHED in the ICUS0ICRSTS register are set to 0 and respectively
	CSEC_WRITE_PROTECTION	If this flag is set, rewriting and erasing of the key are disabled. This flag cannot be cleared once it has been set
	CSEC_VERIFY_ONLY	If this flag is set, the associated memory key slot will be treated as invalid by the CMD_GENERATE_MAC command (only valid for CMD_VERIFY_MAC). The VERIFY_ONLY attribute has no effect if KEY_USAGE == 0
Description	Csec Key Flag 정보.	

4.1.3.6 Csec_StatusType

Name	Csec_StatusType	
Type	Enumeration	
	CSEC_BUSY	While command execution is started
	CSEC_SECURE_BOOT	Checking the validity of BOOT_MAC_KEY after CMD_SECURE_BOOT is executed
	CSEC_BOOT_INIT	CMD_SECURE_BOOT is executed without BOOT_MAC and when registration of BOOT_MAC is completed
	CSEC_BOOT_FINISHED	* SECURE_BOOT = 1, BOOT_FINISHED = 0, and BOOT_OK = 1 and when CMD_BOOT_OK or CMD_BOOT_FAILURE is executed. * CMD_SECURE_BOOT is executed and if its value does not match that of BOOT_MAC * CMD_SECURE_BOOT is executed without BOOT_MAC and if BOOT_MAC_KEY is successfully registered * CMD_SECURE_BOOT is executed and if ERC_KEY_NOT_AVAILABLE, ERC_MEMORY_FAILURE, or ERC_GENERAL_ERROR occurs during the execution of the command * CMD_CANCEL is issued during execution of CMD_SECURE_BOOT and processing of the command is cancelled
	CSEC_BOOT_OK	Execution of CMD_SECURE_BOOT is successfully completed and comparison with BOOT_MAC registered
	CSEC_RND_INIT	Execution of CMD_INIT_RNG is successfully completed
	CSEC_EXT_DEBUGGER	A debug interface is used
	CSEC_INT_DEBUGGER	Execution of CMD_DEBUG is successfully completed

Description

Csec 내부 상태 정보.

Csec_GetStatus API 를 통해 CSEC 내부 상태 정보를 확인할 수 있다.

4.2 Functions

4.2.1 Autron HSM 프레임워크

4.2.1.1 Hsm_EnableSecureBoot

※ SPC58x HSM 프레임워크 only

Function Name	Hsm_EnableSecureBoot	
Syntax	Hsm_ErrType Hsm_EnableSecureBoot(uint32 ulSecureBootAddress, uint32 ulSecureBootSize)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	ulSecureBootAddress	Secure Boot 를 수행할 flash 영역의 시작주소
	ulSecureBootSize	Secure Boot 를 수행할 flash 영역의 시작 주소로부터의 크기
Return Value	Hsm_ErrType	함수 수행 성공/실패 정보
Description	Secure Boot 는 매번 부팅시마다 flash 의 지정된 영역의 지정된 크기의 위변조를 메시지 인증(MAC) 기반으로 확인하는 기능이다. HSM 프레임워크의 Secure Boot 기능이 비활성화 되어 있는 경우, 해당 API 호출을 통해 flash의 지정된 영역을 대상으로 Secure Boot 기능을 활성화시킨다. Secure Boot 기능 활성화 성공시 MCU 가 reset 된다. 한 번 활성화된 Secure Boot 기능은 비활성화 및 지정 영역 변경이 불가능하므로 사용에 주의를 기울여야 한다.	

4.2.1.2 Hsm_GetSecureBootStatus

※ SPC58x / RH850 F1KM ICUM HSM 프레임워크 only

Function Name	Hsm_GetSecureBootStatus	
Syntax	Hsm_ErrType Hsm_GetSecureBootStatus(HSM_SecureBootStatusType* pSecureBootStatus)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (out)	pSecureBootStatus	Secure Boot 상태 확인 변수 포인터 - HSM_SecureBootStatusType 참조
Return Value	Hsm_ErrType	함수 수행 성공/실패 정보
Description	Secure Boot 수행 결과를 확인	

4.2.1.3 Hsm_CustomIauExportData

※ SPC58x HSM 프레임워크 & IAU 제어기 only

Function Name	Hsm_CustomIauExportData	
Syntax	Hsm_ErrType Hsm_CustomIauExportData(boolean blAsync, Hsm_CallbackPtr pCallback, Hsm_DataType ddDataType, uint32 ulDataLength, uint8* pRawData)	
Sync/Async	Sync / Async	
Reentrancy	Non Re-entrant	
Parameters (In)	blAsync	sync / async 동작 여부 - true : Async 동작 (pCallback 인자 입력 필요) - false : Sync 동작
	pCallback	Async 요청 처리 완료시 호출될 callback 함수 포인터 - Sync 동작시 미사용

	ddDataType	추출하고자 하는 데이터 포맷 정보 - Hsm_DataType 참조
	ulDataLength	추출하고자 하는 데이터 길이
Parameters (out)	pRawData	추출하고자 하는 데이터 저장 버퍼
Return Value	Hsm_ErrType	함수 수행 성공/실패 정보
Description	추출이 허용되는 데이터 포맷에 대해 해당하는 HSM 내부 데이터를 리턴	

4.2.1.4 Hsm_CustomlauPinningCheck

※ SPC58x HSM 프레임워크 & IAU 제어기 only

Function Name	Hsm_CustomlauPinningCheck	
Syntax	Hsm_ErrType Hsm_CustomlauPinningCheck(boolean blAsync, Hsm_CallbackPtr pCallback, boolean* pPinningCheckOk)	
Sync/Async	Sync / Async	
Reentrancy	Non Re-entrant	
Parameters (in)	blAsync	sync / async 동작 여부 - true : Async 동작 (pCallback 인자 입력 필요) - false : Sync 동작
	pCallback	Async 요청 처리 완료시 호출될 callback 함수 포인터 - Sync 동작시 미사용
Parameters (out)	pPinningCheckOk	Pinning Check 성공/실패 여부 포인터
Return Value	Hsm_ErrType	함수 수행 성공/실패 정보
Description	함수 호출시 계산된 HSM 내부 저장 RootCA 인증서의 해시값과 사전에 주입된 RootCA 인증서의 해시값을 비교, 일치 여부 리턴	

4.2.1.5 Hsm_CustomlauGenSeedSc

※ SPC58x HSM 프레임워크 & IAU 제어기 only

Function Name	Hsm_CustomlauGenSeedSc	
Syntax	Hsm_ErrType Hsm_CustomlauGenSeedSc(boolean blAsync, Hsm_CallbackPtr pCallback, uint32 ullIterationCount, uint8* pConString, uint32 ulConStringLength, Hsm_DataIDType ddHsmAesKeyID, uint8* pSeecSc)	
Sync/Async	Sync / Async	
Reentrancy	Non Re-entrant	
Parameters (in)	blAsync	sync / async 동작 여부 - true : Async 동작 (pCallback 인자 입력 필요) - false : Sync 동작
	pCallback	Async 요청 처리 완료시 호출될 callback 함수 포인터 - Sync 동작시 미사용
	ullIterationCount	해시 반복 횟수
	pConString	연접 스트링 버퍼 포인터 - 크기 ulConStringLength
	ulConStringLength	연접 스트링 길이
	ddHsmAesKeyID	생성된 SEED_SC 를 암호화 할 AES Key ID - Hsm_CustomlauKdf 함수 호출시 동일한 AES Key ID 사용
Parameters (out)	pSeecSc	암호화된 SEED_SC 가 저장될 버퍼 포인터
Return Value	Hsm_ErrType	함수 수행 성공/실패 정보
Description	연접 스트링에 HSM 내부 저장 Master Seed 을 연접한 스트링을	

ullterationCount 만큼 SHA256 해시한 결과값을 입력받은 AES 암호화하여 리턴

4.2.1.6 Hsm_CustomlauKdf

※ SPC58x HSM 프레임워크 & IAU 제어기 only

Function Name	Hsm_CustomlauKdf	
Syntax	Hsm_ErrType Hsm_CustomlauKdf(boolean blAsync, Hsm_CallbackPtr pCallback, uint8* pRandomNumber, uint8* pTokenID, uint8* pEncSeed, Hsm_DataIDType ddHsmAesKeyID, uint8* pAuthKey, uint8* pEncKey)	
Sync/Async	Sync / Async	
Reentrancy	Non Re-entrant	
Parameters (in)	blAsync	sync / async 동작 여부 - true : Async 동작 (pCallback 인자 입력 필요) - false : Sync 동작
	pCallback	Async 요청 처리 완료시 호출될 callback 함수 포인터 - Sync 동작시 미사용
	pRandomNumber	난수 버퍼 포인터 - 크기 32 byte
	pTokenID	TokenID 버퍼 포인터 - 크기 8 byte
	pEncSeed	암호화된 Seed 버퍼 포인터 - 크기 32 byte
	ddHsmAesKeyID	암호화된 Seed를 복호화 하기 위한 AES Key ID
Parameters (out)	pAuthKey	KDF 결과값 상위 16 byte 저장하기 위한 버퍼 포인터
	pEncKey	KDF 결과값 하위 16 byte 저장하기 위한 버퍼 포인터
Return Value	Hsm_ErrType	함수 수행 성공/실패 정보
Description	입력받은 난수, 복호화된 SEED 값(Enc(SEED) or Enc(SEED_SC))을 AES 복호화), 입력받은 Token 을 SHA256 해시한 결과값을 상위, 하위 16 바이트 분리 리턴	

4.2.2 RH850 F1K 계열 ICUS

4.2.2.1 Icus_EncEcb

※ RH850 F1K 계열 MCU only

Function Name	Icus_EncEcb	
Syntax	Icus_ErrType Icus_EncEcb(Icus_KeyIDType ddKeyId, uint32 ulNumBlocks, const uint8* pPlainBlock, uint8* pCipherBlock)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ddKeyId	AES ECB 모드 암호화시 사용할 Key ID 사용 가능 Key ID : KEY_<n>, RAM_KEY
	ulNumBlocks	입력에 사용되는 128-bit 평문 블록 개수
	pPlainBlock	평문 블록 버퍼 포인터
Parameters (out)	pCipherBlock	암호문 블록 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 ECB(Electronic Code Block) 모드 암호화 수행 평문 : 128-bit 배수의 블록 암호문 : 128-bit 배수의 블록 입출력 데이터는 반드시 128-bit 의 배수여야 한다 ※ Icus_EncEcb API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmSymBlockEncrypt 를 통해 사용한다	

4.2.2.2 Icus_DecEcb

※ RH850 F1K 계열 MCU only

Function Name	Icus_DecEcb	
Syntax	Icus_ErrType Icus_DecEcb(Icus_KeyIDType ddKeyId, uint32 uiNumBlocks, const uint8* pCipherBlock, uint8* pPlainBlock)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ddKeyId	AES ECB 모드 복호화시 사용할 Key ID 사용 가능 Key ID : KEY_<n>, RAM_KEY
	uiNumBlocks	입력에 사용되는 128-bit 평문 블록 개수
	pCipherBlock	암호문 블록 버퍼 포인터
Parameters (out)	pPlainBlock	평문 블록 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 ECB(Electronic Code Block) 모드 복호화 수행 암호문 : 128-bit 배수의 블록 평문 : 128-bit 배수의 블록 입출력 데이터는 반드시 128-bit 의 배수여야 한다 ※ Icus_DecEcb API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmSymBlockDecrypt 를 통해 사용한다	

4.2.2.3 Icus_EncCbc

※ RH850 F1K 계열 MCU only

Function Name	Icus_EncCbc	
Syntax	Icus_ErrType Icus_EncCbc(Icus_KeyIDType ddKeyId, uint32 uiNumBlocks, const uint8* pPlainBlock, uint8* pCipherBlock, const uint8* pIVBlock)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ddKeyId	AES CBC 모드 암호화시 사용할 Key ID 사용 가능 Key ID : KEY_<n>, RAM_KEY
	uiNumBlocks	입력에 사용되는 128-bit 평문 블록 개수
	pPlainBlock	평문 블록 버퍼 포인터
	pIVBlock	Initialization Vector 버퍼(128-bit) 포인터
Parameters (out)	pCipherBlock	암호문 블록 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 CBC(Cipher Block Chaining) 모드 암호화 수행 평문 : 128-bit 배수의 블록 암호문 : 128-bit 배수의 블록 입출력 데이터는 반드시 128-bit 의 배수여야 한다 ※ Icus_EncCbc API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmSymEncrypt 를 통해 사용한다	

4.2.2.4 Icus_DecCbc

※ RH850 F1K 계열 MCU only

Function Name	Icus_DecCbc	
Syntax	Icus_ErrType Icus_DecCbc (Icus_KeyIDType ddKeyId, uint32 uiNumBlocks, const uint8* pCipherBlock, uint8* pPlainBlock, const uint8* pIVBlock)	
Sync/Async	Sync	

Reentrancy	Re-entrant	
Parameters (in)	ddKeyId	AES CBC 모드 복호화시 사용할 Key ID 사용 가능 Key ID : KEY_<n>, RAM_KEY
	ulNumBlocks	입력에 사용되는 128-bit 평문 블록 개수
	pCipherBlock	암호문 블록 버퍼 포인터
	pIVBlock	Initialization Vector 버퍼(128-bit) 포인터
Parameters (out)	pPlainBlock	평문 블록 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 CBC(Cipher Block Chaining) 모드 복호화 수행 암호문 : 128-bit 배수의 블록 평문 : 128-bit 배수의 블록 입출력 데이터는 반드시 128-bit 의 배수여야 한다 ※ Icus_DecCbc API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmSymDecrypt 를 통해 사용한다	

4.2.2.5 Icus_GenerateMac

※ RH850 F1K 계열 MCU only

Function Name	Icus_GenerateMac	
Syntax	Icus_ErrType Icus_GenerateMac(Icus_KeyIDType ddKeyId, uint32 ulTotalMsgBitLength, const uint8* pMsgBlock, uint32 ulMacByteLength, uint8* pMac)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ddKeyId	CMAC-AES 생성 수행시 사용할 Key ID 사용 가능 Key ID : KEY_<n>, RAM_KEY
	ulTotalMsgBitLength	메세지 데이터 전체 bit 단위 길이
	pMsgBlock	메세지 데이터 버퍼 포인터
	ulMacByteLength	원하는 결과 MAC 값의 byte 단위 길이
Parameters (out)	pMac	MAC 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 기반 Cipher-based Message Authentication Code 생성 수행 Icus_GenerateMac api 는 blocking 기반 동작으로 watchdog timeout 등의 문제를 피하고자 할 경우 Icus_GenerateMacStart/Update/Finish 를 사용한다 ※ Icus_GenerateMac API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmMacGenerate 를 통해 사용한다	

4.2.2.6 Icus_GenerateMacStart

※ RH850 F1K 계열 MCU only

Function Name	Icus_GenerateMacStart	
Syntax	Icus_ErrType Icus_GenerateMacStart(Icus_KeyIDType ddKeyId, uint32 ulTotalMsgBitLength)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ddKeyId	CMAC-AES 생성 수행시 사용할 Key ID 사용 가능 Key ID : KEY_<n>, RAM_KEY
	ulTotalMsgBitLength	메세지 데이터 전체 bit 단위 길이
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 기반 Cipher-based Message Authentication Code 생성 수행	

Streaming 방식(Start→Update(반복가능)→Finish) 으로 CMAC 생성 수행
Start : Key 정보와 메시지 데이터 전체 bit 단위 길이 입력
CMAC 생성 완료를 위해서는 이어서 Icus_GenerateMacUpdate, Icus_GenerateMacFinish 를 수행 해야 한다
※ Icus_GenerateMacStart API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmMacGenerate 를 통해 사용한다

4.2.2.7 Icus_GenerateMacUpdate

※ RH850 F1K 계열 MCU only

Function Name	Icus_GenerateMacUpdate	
Syntax	Icus_ErrType Icus_GenerateMacUpdate(const uint8* pMsgBlock, uint32 ulInputMsgBitLength)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	pMsgBlock	메세지 데이터 버퍼 포인터
	ulInputMsgBitLength	메세지 데이터 bit 단위 길이
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 기반 Cipher-based Message Authentication Code 생성 수행 Streaming 방식(Start→Update(반복가능)→Finish) 으로 CMAC 생성 수행 Update : 분할된 메세지 데이터를 입력, 반복 호출 가능 ex) 256 bit * n 회 + 63 bit ※ Icus_GenerateMacUpdate API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmMacGenerate 를 통해 사용한다	

4.2.2.8 Icus_GenerateMacFinish

※ RH850 F1K 계열 MCU only

Function Name	Icus_GenerateMacFinish	
Syntax	Icus_ErrType Icus_GenerateMacFinish(uint8* pMac, uint32 ulMacBitLength□)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ulMacByteLength	원하는 결과 MAC 값의 byte 단위 길이
Parameters (out)	pMac	MAC 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 기반 Cipher-based Message Authentication Code 생성 수행 Streaming 방식(Start→Update(반복가능)→Finish) 으로 CMAC 생성 수행 Finish : Start, Update 수행의 결과물인 MAC 값 추출 ※ Icus_GenerateMacFinish API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmMacGenerate 를 통해 사용한다	

4.2.2.9 Icus_VerifyMac

※ RH850 F1K 계열 MCU only

Function Name	Icus_VerifyMac	
Syntax	Icus_ErrType Icus_VerifyMac(Icus_KeyIDType ddKeyId, uint32 ulTotalMsgBitLength, const uint8* pMsgBlock, uint32 ulMacBitLength□, const uint8* pMac, boolean* pBtResult)	
Sync/Async	Sync	
Reentrancy	Re-entrant	

Parameters (in)	ddKeyId	CMAC-AES 검증 수행시 사용할 Key ID 사용 가능 Key ID : KEY_<n>, RAM_KEY
	ulTotalMsgBitLength	메세지 데이터 전체 bit 단위 길이
	pMsgBlock	메세지 데이터 버퍼 포인터
	ulMacByteLength	원하는 결과 MAC 값의 byte 단위 길이
	pMac	MAC 버퍼 포인터
Parameters (out)	pblResult	검증 결과 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 기반 Cipher-based Message Authentication Code 검증 수행 Icus_VerifyMac api 는 blocking 기반 동작으로 watchdog timeout 등의 문제를 피하고자 할 경우 Icus_VerifyMacStart/Update/Finish 를 사용한다 ※ Icus_VerifyMac API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmMacVerify 를 통해 사용한다	

4.2.2.10 Icus_VerifyMacStart

※ RH850 F1K 계열 MCU only

Function Name	Icus_VerifyMacStart	
Syntax	Icus_ErrType Icus_VerifyMacStart(Icus_KeyIDType ddKeyId, uint32 ulTotalMsgBitLength)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ddKeyId	CMAC-AES 검증 수행시 사용할 Key ID 사용 가능 Key ID : KEY_<n>, RAM_KEY
	ulTotalMsgBitLength	메세지 데이터 전체 bit 단위 길이
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 기반 Cipher-based Message Authentication Code 검증 수행 Streaming 방식(Start->Update(반복가능)->Finish) 으로 CMAC 검증 수행 Start : Key 정보와 메세지 데이터 전체 bit 단위 길이 입력 CMAC 검증 완료를 위해서는 이어서 Icus_VerifyMacUpdate, Icus_VerifyMacFinish 를 수행 해야 한다 ※ Icus_VerifyMacStart API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmMacVerify 를 통해 사용한다	

4.2.2.11 Icus_VerifyMacUpdate

※ RH850 F1K 계열 MCU only

Function Name	Icus_VerifyMacUpdate	
Syntax	Icus_ErrType Icus_VerifyMacUpdate(const uint8* pMsgBlock, uint32 ulInputMsgBitLength)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	pMsgBlock	메세지 데이터 버퍼 포인터
	ulInputMsgBitLength	메세지 데이터 bit 단위 길이
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 기반 Cipher-based Message Authentication Code 검증 수행 Streaming 방식(Start->Update(반복가능)->Finish) 으로 CMAC 검증 수행 Update : 분할된 메세지 데이터를 입력, 반복 호출 가능 ex) 256 bit * n 회 + 63 bit ※ Icus_VerifyMacUpdate API 의 직접 사용은 지양하고 Autosar Csm 모듈의	

CsmMacVerify 를 통해 사용한다

4.2.2.12 Icus_VerifyMacFinish

※ RH850 F1K 계열 MCU only

Function Name	Icus_VerifyMacFinish	
Syntax	Icus_ErrType Icus_VerifyMacFinish(uint32 ulMacBitLength, const uint8* pMac, boolean* pblResult)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ulMacBitLength	검증에 사용할 MAC 값의 bit 단위 길이
	pMac	MAC 버퍼 포인터
Parameters (out)	pblResult	검증 결과 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	AES-128 기반 Cipher-based Message Authentication Code 검증 수행 Streaming 방식(Start→Update(반복가능)→Finish) 으로 CMAC 검증 수행 Finish : Start, Update 수행의 결과물인 MAC 값 추출 ※ Icus_VerifyMacFinish API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmMacVerify 를 통해 사용한다	

4.2.2.13 Icus_InitRnd

※ RH850 F1K 계열 MCU only

Function Name	Icus_InitRnd	
Syntax	Icus_ErrType Icus_InitRnd(void)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Return Value	Icus_ErrType	기능 수행 결과
Description	TRNG 로 PRNG 의 seed 를 공급한다 TRNG 의 AIS31 표준에 따른 난수성을 보장하기 위해 최초 CMD_INIT_RNG 수행시 상당한 수행시간(최대 $2248/f_{CLK} + 120000 \mu s$) 이 걸린다 reset 이후 CMD_RND 사용 전에 반드시 CMD_INIT_RNG 가 한 번 호출되었어야 한다 ※ Icus_InitRnd API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmRandomGenerate 를 통해 사용한다	

4.2.2.14 Icus_ExtendSeed

※ RH850 F1K 계열 MCU only

Function Name	Icus_ExtendSeed	
Syntax	Icus_ErrType Icus_ExtendSeed (uint32 ulSeed)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ulSeed	128-bit entropy
Return Value	Icus_ErrType	기능 수행 결과
Description	128-bit 입력 entropy 로 PRNG 의 seed 를 확장한다 reset 이후 CMD_EXTEND_SEED 사용 전에 반드시 CMD_INIT_RNG 가 한 번 호출되었어야 한다 ※ Icus_ExtendSeed API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmRandomSeed 를 통해 사용한다	

4.2.2.15 Icus_Rnd

※ RH850 F1K 계열 MCU only

Function Name	Icus_Rnd	
Syntax	Icus_ErrType Icus_Rnd(uint32* pRnd)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	pRnd	난수 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	128-bit 난수를 생성한다 동시에 내부 PRNG 의 seed 를 업데이트한다 reset 이후 CMD_RND 사용 전에 반드시 CMD_INIT_RNG 가 한 번 호출되었어야 한다 ※ Icus_Rnd API 의 직접 사용은 지양하고 Autosar Csm 모듈의 CsmRandomGenerate 를 통해 사용한다	

4.2.2.16 Icus_GenerateProtectedKey

※ RH850 F1K 계열 MCU only

Function Name	Icus_GenerateProtectedKey	
Syntax	Icus_ErrType Icus_GenerateProtectedKey(Icus_KeyIDType ddKeyId, const uint8* pKey, Icus_KeyIDType ddAuthId, const uint8* pAuthKey, uint32 ulCounter, uint8 ucKeyFlag, Icus_ProtectedKey* pProtectedKey)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ddKeyId	등록/업데이트 대상 Key ID 사용 가능 Key ID : MASTER_ECU_KEY, BOOT_MAC_KEY, BOOT_MAC, KEY_<n>, RAM_KEY
	pKey	등록/업데이트 대상 버퍼 포인터
	ddAuthId	등록/업데이트 대상 Key 의 AuthKey ID
	pAuthKey	AuthKey 버퍼 포인터
	ulCounter	등록/업데이트 대상 Key 의 신규 counter 값
	ucKeyFlag	등록/업데이트 대상 Key 의 flag 특성 정보
Parameters (out)	pProtectedKey	M1, M2, M3 : 입력된 정보 기반으로 생성된 message M4, M5 : 미사용
Return Value	Icus_ErrType	기능 수행 결과
Description	CMD_LOAD_KEY 호출시 input 인 M1, M2, M3 를 생성하기 위한 API 입력 받은 ddKeyId, pKey, ddAuthId, pAuthKey, ulCounter, ucKeyFlag 기반으로 pProtectedKey 멤버 중 M1, M2, M3 를 생성한다 M1, M2, M3 계산과정에 필요한 K1, K2 는 Miyaguchi-Preneel compression algorithm 의 KDF(Key Derivation Function) 로 내부적으로 생성하여 적용 K1 = KDF(KEY[AuthID], KEY_UPDATE_ENC_C) K2 = KDF(KEY[AuthID], KEY_UPDATE_MAC_C) M1(128 bits) = UID(120 bit) + ID(4 bit) + AuthID(4 bit) M2(256 bits) = ENC[CBC, K1, IV = 0] (Counter(28 bit) + KEY_FLAG(6 bit) + "0 ... 0"(94 bit) + KEY_ID(128 bit)) M3(128 bits) = CMAC[K2](M1 M2)	

등록/업데이트 대상 Key ID 와 AuthKey ID 관계

Key ID AuthKey ID

MASTER_ECU_KEY : MASTER_ECU_KEY

BOOT_MAC_KEY : MASTER_ECU_KEY, BOOT_MAC_KEY

BOOT_MAC : MASTER_ECU_KEY, BOOT_MAC_KEY

KEY_<n> : MASTER_ECU_KEY, KEY_<n> (*2)

RAM_KEY : SECRET_KEY (*1), KEY_<n>

Note 1. Only used when registering M1 to M3 that are acquired by the CMD_EXPORT_RAM_KEY command

Note 2. Only used when ID = AuthID

Counter 는 28 bit 길이의 값으로 replay attack 을 방지하고자 사용된다

Key 등록시 counter 값은 ICUS 내부에 저장되며 Key 업데이트 시 이전에 저장된 값보다 반드시 큰 값이 제공되어야 한다

미등록된 Key 의 counter 는 0 이기 때문에 최초 Key 등록시에는 반드시 1 보다 큰 값이 제공되어야 한다

KeyFlag 는 등록/업데이트되는 Key 의 특성을 설정

Icus_KeyFlagType 항목 설명을 참조한다

4.2.2.17 Icus_VerifyProtectedKey

※ RH850 F1K 계열 MCU only

Function Name	Icus_VerifyProtectedKey	
Syntax	Icus_ErrType Icus_VerifyProtectedKey(Icus_KeyIDType ddKeyId, const uint8* pKey, Icus_KeyIDType ddAuthId, uint32 ulCounter, Icus_ProtectedKey* pProtectedKey, boolean* pResult)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ddKeyId	검증 대상 Key ID
	pKey	검증 대상 버퍼 포인터
	ddAuthId	검증 대상 Key 의 AuthKey ID
	ulCounter	검증 대상 Key 의 등록된 counter 값
	pProtectedKey	M1, M2, M3 : 미사용 M4, M5 : 검증에 사용
Parameters (out)	pResult	입력받은 M4, M5 가 valid 할 경우 ICUS_TRUE, invalid 할 경우 ICUS_FALSE 리턴
Return Value	Icus_ErrType	기능 수행 결과
Description	CMD_LOAD_KEY 호출후 output 인 M4, M5 를 검증하기 위한 API 입력 받은 ddKeyId, pKey, ddAuthId, ulCounter 기반으로 M4, M5 를 생성하여 입력받은 pProtectedKey 의 M4, M5 를 검증한다. M4, M5 계산과정에 필요한 K3, K4 는 Miyaguchi-Preneel compression algorithm 의 KDF(Key Derivation Function) 로 내부에서 생성하여 적용 K3 = KDF(KEY[ID], KEY_UPDATE_ENC_C) K4 = KDF(KEY[ID], KEY_UPDATE_MAC_C)	

M4(256 bits) = UID(120 bit) + ID(4 bit) + AuthID(4 bit) + ENC[ECB, K3](Counter(28 bit) + "1" + "0 ... 0"(99 bit))
M5(128 bits) = CMAC[K4](M4)

4.2.2.18 Icus_LoadKey

※ RH850 F1K 계열 MCU only

Function Name	Icus_LoadKey	
Syntax	Icus_ErrType Icus_LoadKey(Icus_KeyIDType ddKeyId, Icus_ProtectedKey* pProtectedKey)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ddKeyId	등록/업데이트 대상 Key ID 사용 가능 Key ID : SECRET_KEY, MASTER_ECU_KEY, BOOT_MAC_KEY, BOOT_MAC, KEY_<n>, RAM_KEY
	pProtectedKey	M1, M2, M3 는 input, M4, M5 는 output
Return Value	Icus_ErrType	기능 수행 결과
Description	유도된 M1, M2, M3 로 ICUS 내부 Key 를 등록/업데이트 한다 pProtectedKey 의 M4, M5 는 Key 등록 절차를 검증 (Icus_VerifyProtectedKey) 하는데 쓰인다	

4.2.2.19 Icus_LoadPlainKey

※ RH850 F1K 계열 MCU only

Function Name	Icus_LoadPlainKey	
Syntax	Icus_ErrType Icus_LoadPlainKey(const uint8* pInput)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	pInput	평문 형태의 16 바이트 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	평문 Key 값을 ICU_RAM_KEY 에 등록한다	

4.2.2.20 Icus_ExportRamKey

※ RH850 F1K 계열 MCU only

Function Name	Icus_ExportRamKey	
Syntax	Icus_ErrType Icus_ExportRamKey(Icus_ProtectedKey* pProtectedKey)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (out)	pProtectedKey	추출된 M1, M2, M3, M4, M5 를 저장할 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	CMD_LOAD_PLAIN_KEY 로 RAM_KEY 에 등록한 데이터를 SECRET_KEY 기반 으로 M1, M2, M3, M4, M5 를 추출한다 RAM_KEY slot 이 비어 있거나 RAM_KEY 를 CMD_LOAD_KEY 로 등록한 경우 에러가 발생한다	

4.2.2.21 Icus_SecureBoot

※ RH850 F1K 계열 MCU only

Function Name	Icus_SecureBoot	
Syntax	Icus_ErrType Icus_SecureBoot(const uint32 ulByteSize, const uint8* pAddress)	

Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	ulByteSize	Secure Boot 대상 byte 크기(0 보다 큰 값)
	pAddress	Secure Boot 대상 시작 주소
Return Value	Icus_ErrType	기능 수행 결과
Description	Secure Boot 기능을 수행한다 BOOT_MAC_KEY, BOOT_MAC 이 등록된 상태에서는 pAddress 주소부터 ulByteSize 블록의 MAC 값을 계산하여 BOOT_MAC 값과 일치하는지 비교한다 BOOT_MAC slot 이 비어 있는 경우 생성된 MAC 값을 BOOT_MAC slot 에 등록한다	

4.2.2.22 Icus_BootFailure

※ RH850 F1K 계열 MCU only

Function Name	Icus_BootFailure	
Syntax	Icus_ErrType Icus_BootFailure(void)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Return Value	Icus_ErrType	기능 수행 결과
Description	ICUS 내부 상태 값 중 BOOT_FINISHED flag 를 1 로, BOOT_OK flag 를 0 으로 셋팅한다 ICUS 내부 상태가 SECURE_BOOT = 1 & BOOT_FINISHED = 0 & BOOT_OK = 1 인 조건을 만족해야 한다	

4.2.2.23 Icus_BootOk

※ RH850 F1K 계열 MCU only

Function Name	Icus_BootOk	
Syntax	Icus_ErrType Icus_BootOk(void)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Return Value	Icus_ErrType	기능 수행 결과
Description	ICUS 내부 상태 값 중 BOOT_FINISHED flag 를 1 로 셋팅한다 ICUS 내부 상태가 SECURE_BOOT = 1 & BOOT_FINISHED = 0 & BOOT_OK = 1 인 조건을 만족해야 한다	

4.2.2.24 Icus_GetStatus

※ RH850 F1K 계열 MCU only

Function Name	Icus_GetStatus	
Syntax	Icus_ErrType Icus_GetStatus(Icus_StatusType* pStatus)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (out)	pStatus	ICUS 내부 상태값 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	ICUS 내부 상태 값인 SREG 값을 얻는다	

4.2.2.25 Icus_GetUID

※ RH850 F1K 계열 MCU only

Function Name	Icus_GetUID
----------------------	-------------

Syntax	Icus_ErrType Icus_GetUID(uint8* pUID)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (out)	pUID	UID + SREG 버퍼(128 bits) 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	Device 의 UID(120 bit), SREG(8 bit) 를 추출한다	

4.2.2.26 Icus_Cancel

※ RH850 F1K 계열 MCU only

Function Name	Icus_Cancel	
Syntax	Icus_ErrType Icus_Cancel(void)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Return Value	Icus_ErrType	기능 수행 결과
Description	현재 진행중인 command 를 취소한다	

4.2.2.27 Icus_Debug

※ RH850 F1K 계열 MCU only

Function Name	Icus_Debug	
Syntax	Icus_ErrType Icus_Debug(const uint8* pMasterKey)	
Sync/Async	Sync	
Reentrancy	Re-entrant	
Parameters (in)	pMasterKey	MASTER_ECU_KEY 로 등록된 Key 의 raw key 값 버퍼 포인터
Return Value	Icus_ErrType	기능 수행 결과
Description	ICUS 에 저장된 모든 Non-volatile memory Key, RAM_KEY 를 지운다. 개별 Key 삭제는 불가능	

4.2.3 S32K 계열 CSEC

4.2.3.1 Csec_Init

※ S32K 계열 MCU only

Function Name	Csec_Init	
Syntax	Csec_ErrType Csec_Init(void)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Return Value	Csec_ErrType	기능 수행 결과
Description	This function initializes the CSEC driver. If the hardware is not initialized, the function shall report CSEC_E_HW_UNINIT to the DET Hardware must be initialized	

4.2.3.2 Csec_EncEcb

※ S32K 계열 MCU only

Function Name	Csec_EncEcb	
Syntax	Csec_ErrType Csec_EncEcb(Csec_KeyIDType keyID, uint32 plainTextLength, const uint8* plainTextPtr, uint8* cipherTextPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	

Parameters (in)	keyID	KeyID used to perform the cryptographic operation
	plainTextLength	Number of bytes of plain text message to be encrypted; should be smaller than 1048560 bytes
	plainTextPtr	Pointer to the plain text buffer to be encrypted
Parameters (out)	cipherTextPtr	Pointer to the cipher text buffer. The buffer shall have the same size as the plain text buffer
Return Value	Csec_ErrType	기능 수행 결과
Description	This function performs the AES-128 encryption in ECB mode of the input plaintext buffer. It operates under the assumption that the plaintext length is a multiple of 16 bytes. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.3 Csec_DecEcb

※ S32K 계열 MCU only

Function Name	Csec_DecEcb	
Syntax	Csec_ErrType Csec_DecEcb(Csec_KeyIDType keyID, uint32 cipherTextLength, const uint8* cipherTextPtr, uint8* plainTextPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	keyID	KeyID used to perform the cryptographic operation
	cipherTextPtr	Pointer to the cipher text buffer
	cipherTextLength	Number of bytes of cipher text message to be decrypted; should be smaller than 1048560 bytes
Parameters (out)	plainTextPtr	Pointer to the plain text buffer. The buffer shall have the same size as the cipher text buffer
Return Value	Csec_ErrType	기능 수행 결과
Description	This function performs the AES-128 decryption in ECB mode of the input ciphertext buffer. It operates under the assumption that the ciphertext length is a multiple of 16 bytes. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.4 Csec_EncCbc

※ S32K 계열 MCU only

Function Name	Csec_EncCbc	
Syntax	Csec_ErrType Csec_EncCbc(Csec_KeyIDType keyID, uint32 plainTextLength, const uint8* plainTextPtr, uint8* cipherTextPtr, const uint8* ivPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	keyID	KeyID used to perform the cryptographic operation
	plainTextLength	Number of bytes of plain text message to be encrypted; should be smaller than 1048560 bytes
	plainTextPtr	Pointer to the plain text buffer

	ivPtr	Pointer to the initialization vector buffer
Parameters (out)	cipherTextPtr	Pointer to the cipher text buffer. The buffer shall have the same size as the plain text buffer
Return Value	Csec_ErrType	기능 수행 결과
Description	This function performs the AES-128 encryption in CBC mode of the input plaintext buffer. It operates under the assumption that the plaintext length is a multiple of 16 bytes. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.5 Csec_DecCbc

※ S32K 계열 MCU only

Function Name	Csec_DecCbc	
Syntax	Csec_ErrType Csec_DecCbc(Csec_KeyIDType keyID, uint32 cipherTextLength, const uint8* cipherTextPtr, uint8* plainTextPtr, const uint8* ivPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	keyID	KeyID used to perform the cryptographic operation
	cipherTextPtr	Pointer to the cipher text buffer
	cipherTextLength	Number of bytes of cipher text message to be decrypted; should be smaller than 1048560 bytes
	ivPtr	Pointer to the initialization vector buffer
Parameters (out)	plainTextPtr	Pointer to the plain text buffer. The buffer shall have the same size as the cipher text buffer
Return Value	Csec_ErrType	기능 수행 결과
Description	This function performs the AES-128 decryption in CBC mode of the input ciphertext buffer. It operates under the assumption that the ciphertext length is a multiple of 16 bytes. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.6 Csec_GenerateMac

※ S32K 계열 MCU only

Function Name	Csec_GenerateMac	
Syntax	Csec_ErrType Csec_GenerateMac(Csec_KeyIDType keyID, const uint8* dataPtr, uint32 dataLength, uint8* macPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	keyID	KeyID used to perform the cryptographic operation
	dataPtr	Pointer to the message buffer
	dataLength	Number of bits of message on which CMAC is to be computed

Parameters (out)	macPtr	Pointer to the 128-bit buffer containing the result of the CMAC generation
Return Value	Csec_ErrType	기능 수행 결과
Description	This function calculates the MAC of a given message based on CMAC with the 128-bit Advanced Encryption Standard. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.7 Csec_VerifyMac

※ S32K 계열 MCU only

Function Name	Csec_VerifyMac	
Syntax	Csec_ErrType Csec_VerifyMac(Csec_KeyIDType keyID, const uint8* dataPtr, uint32 dataLength, uint8 macLength, const uint8* macRefPtr, boolean* statusPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	keyID	KeyID used to perform the cryptographic operation
	dataPtr	Pointer to the message buffer
	dataLength	Number of bits of message on which CMAC is to be computed
	macLength	umber of bits of the CMAC to be compared. A macLength value of zero indicates that all 128-bits are to be compared
	macRefPtr	ointer to the buffer containing the CMAC to be verified
Parameters (out)	statusPtr	Status of the MAC verification command (true: verification operation passed; false: verification operation failed)
Return Value	Csec_ErrType	기능 수행 결과
Description	This function verifies the MAC of of a given message based on CMAC with the 128-bit Advanced Encryption Standard. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.8 Csec_LoadKey

※ S32K 계열 MCU only

Function Name	Csec_LoadKey	
Syntax	Csec_ErrType Csec_VerifyMac(Csec_KeyIDType keyID, const uint8* dataPtr, uint32 dataLength, uint8 macLength, const uint8* macRefPtr, boolean* statusPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	keyID	The ID of the key to be updated
	protectedKeyPtr	Pointer to the M1 ... M5 structure

Return Value	Csec_ErrType	기능 수행 결과
Description	The function updates an internal key per the SHE specification. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.9 Csec_LoadPlainKey

※ S32K 계열 MCU only

Function Name	Csec_LoadPlainKey	
Syntax	Csec_ErrType Csec_LoadPlainKey(const uint8* keyPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	keyPtr	Pointer to the 128-bit buffer containing the key that needs to be copied in RAM_KEY slot
Return Value	Csec_ErrType	기능 수행 결과
Description	The function updates the RAM key memory slot with a 128-bit plaintext. The key is loaded without the encryption and verification of the key, i.e. the key is handed over in plaintext. A plain key can only be loaded into the RAM_KEY slot. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.10 Csec_ExportRamKey

※ S32K 계열 MCU only

Function Name	Csec_ExportRamKey	
Syntax	Csec_ErrType Csec_ExportRamKey(Csec_ProtectedKey* protectedKeyPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	protectedKeyPtr	Pointer to the M1 ... M5 structure
Return Value	Csec_ErrType	기능 수행 결과
Description	The function exports the RAM_KEY into a format protected by SECRET_KEY. Only keys loaded with the CMD_LOAD_PLAIN_KEY command may be exported. The output messages are compatible with the messages used for LOAD_KEY. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.11 Csec_InitRnd

※ S32K 계열 MCU only

Function Name	Csec_InitRnd	
Syntax	Csec_ErrType Csec_InitRnd(void)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Return Value	Csec_ErrType	기능 수행 결과
Description	This function initializes the seed and derives a key for the PRNG. The	

function must be called before CMD_RND after every power cycle or reset. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes
Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)

4.2.3.12 Csec_ExtendSeed

※ S32K 계열 MCU only

Function Name	Csec_ExtendSeed	
Syntax	Csec_ErrType Csec_ExtendSeed(const uint8* entropyPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	entropyPtr	Pointer to a 128-bit buffer containing the entropy
Return Value	Csec_ErrType	기능 수행 결과
Description	This function extends the seed of the PRNG by compressing the former seed value and the supplied entropy into a new seed which will be used to generate the following random numbers. The random number generator has to be initialized by Csec_InitRnd() before the seed can be extended. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.13 Csec_Rnd

※ S32K 계열 MCU only

Function Name	Csec_Rnd	
Syntax	Csec_ErrType Csec_Rnd(uint8* rndPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (out)	rndPtr	Pointer to a 128-bit buffer where the generated random number is to be stored
Return Value	Csec_ErrType	기능 수행 결과
Description	The function returns a vector of 128 random bits. The random number generator has to be initialized by Csec_InitRnd() before random numbers can be supplied. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.14 Csec_BootFailure

※ S32K 계열 MCU only

Function Name	Csec_BootFailure	
Syntax	Csec_ErrType Csec_BootFailure(void)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Return Value	Csec_ErrType	기능 수행 결과
Description	This function is called during later stages of the boot process to detect a failure. This function does not work for VLRP (Very Low Power) and HSRUN	

(High Speed Run) modes
Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)

4.2.3.15 Csec_BootOk

※ S32K 계열 MCU only

Function Name	Csec_BootOk	
Syntax	Csec_ErrType Csec_BootOk (void)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Return Value	Csec_ErrType	기능 수행 결과
Description	The function is called during later stages of the boot process to mark the successful boot verification. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.16 Csec_BootDefine

※ S32K 계열 MCU only

Function Name	Csec_BootDefine	
Syntax	Csec_ErrType Csec_BootDefine(uint32 bootSize, Csec_BootFlavorType bootMethod)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	bootSize	Number of blocks of 128-bit data to check on boot. Maximum size is 512kBytes
	bootMethod	The boot method
Return Value	Csec_ErrType	기능 수행 결과
Description	The function implements an extension of the SHE standard to define both the user boot size and boot method. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes. During parallel boot operation FLASH_CLK must be the default FIRC_CLK and should not be changed until boot operation is complete Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.17 Csec_GetStatus

※ S32K 계열 MCU only

Function Name	Csec_GetStatus	
Syntax	Csec_StatusType Csec_GetStatus(void)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Return Value	Csec_StatusType	Value of the status register
Description	The function shall return the content of the status register Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.18 Csec_GetUID

※ S32K 계열 MCU only

Function Name	Csec_GetUID	
Syntax	Csec_ErrType Csec_GetUID(const uint8* challengePtr, uint8* sregPtr, uint8* uidPtr, uint8* macPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	challengePtr	Number of blocks of 128-bit data to check on boot. Maximum size is 512kBytes
	sregPtr	Value of the status register
	uidPtr	Pointer to 120-bit buffer where the UID will be stored
	macPtr	Pointer to the 128 bit buffer where the MAC generated over the challenge, UID and status will be stored
Return Value	Csec_ErrType	기능 수행 결과
Description	This function returns the identity (UID) and the value of the status register protected by a MAC over a challenge and the data. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.19 Csec_DbgChal

※ S32K 계열 MCU only

Function Name	Csec_DbgChal	
Syntax	Csec_ErrType Csec_DbgChal(uint8* challengePtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (out)	challengePtr	Pointer to the 128-bit buffer where the challenge data will be stored
Return Value	Csec_ErrType	기능 수행 결과
Description	This function is used to obtain a random number which the user will use, along with the MASTER_ECU_KEY and UID, to generate an authorization request. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.20 Csec_DbgAuth

※ S32K 계열 MCU only

Function Name	Csec_DbgAuth	
Syntax	Csec_ErrType Csec_DbgAuth(const uint8* authorizationPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	authorizationPtr	Pointer to the 128-bit buffer containing the authorization value
Return Value	Csec_ErrType	기능 수행 결과
Description	This function erases all keys (actual and outdated) stored in NVM Memory if the authorization is confirmed by CSec. This function does not work for	

VLRP (Very Low Power) and HSRUN (High Speed Run) modes
Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)

4.2.3.21 Csec_MPCompress

※ S32K 계열 MCU only

Function Name	Csec_MPCompress	
Syntax	Csec_ErrType Csec_MPCompress(const uint8* dataPtr, uint16 messageLength, uint8* compressPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	dataPtr	Pointer to the messages to be compressed. Messages must be pre-processed per SHE specification if they do not already meet the full 128-bit block size requirement
	messageLength	The number of 128-bit messages to be compressed
Parameters (out)	compressPtr	Pointer to the 128 bit buffer storing the compressed data
Return Value	Csec_ErrType	기능 수행 결과
Description	This function accesses a Miyaguchi-Preneel compression feature within the CSEc feature set to compress the given messages. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.22 Csec_Debug

※ S32K 계열 MCU only

Function Name	Csec_Debug	
Syntax	Csec_ErrType Csec_Debug(const uint8* masterEcuKeyPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	masterEcuKeyPtr	Pointer to the raw MASTER_ECU_KEY buffer
Return Value	Csec_ErrType	기능 수행 결과
Description	This function deletes all keys in NVM memory by performing random number generation, UID acquisition and authorization generation. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.23 Csec_GenerateProtectedKey

※ S32K 계열 MCU only

Function Name	Csec_GenerateProtectedKey	
Syntax	Csec_ErrType Csec_GenerateProtectedKey(Csec_KeyIDType keyID, const uint8* keyPtr, Csec_KeyIDType authID, const uint8* authKeyPtr, uint32 counter, uint8 keyFlag, Csec_ProtectedKey* protectedKeyPtr)	
Sync/Async	Sync	

Reentrancy	Non Re-entrant	
Parameters (in)	keyID	Key ID to be registered / updated
	keyPtr	Pointer to the target key buffer to be registered / updated
	authID	AuthKey ID of target key to be registered / updated
	authKeyPtr	Pointer to the AuthKey buffer of target key to be registered / updated
	counter	New counter value of registered /updated key
	keyFlag	Flag information of registered / updated key
Parameters (out)	protectedKeyPtr	Pointer to protected key buffer which has M1, M2 and M3
Return Value	Csec _ErrType	기능 수행 결과
Description	This function generates M1, M2 and M3 based on input Key ID, Key value, AuthKey ID, AuthKey value, Counter and KeyFlag information. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

4.2.3.24 Csec_VerifyProtectedKey

※ S32K 계열 MCU only

Function Name	Csec_VerifyProtectedKey	
Syntax	Csec_ErrType Csec_VerifyProtectedKey(Csec_KeyIDType keyID, const uint8* keyPtr, Csec_KeyIDType authID, uint32 counter, const Csec_ProtectedKey* protectedKeyPtr, boolean* resultPtr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	keyID	Key ID to be registered / updated
	keyPtr	Pointer to the target key buffer to be registered / updated
	authID	AuthKey ID of target key to be registered / updated
	counter	New counter value of registered /updated key
	protectedKeyPtr	Pointer to protected key buffer which has M4 and M5
Parameters (out)	resultPtr	Status of the verification
Return Value	Csec _ErrType	기능 수행 결과
Description	This function verifies M4 and M5 which was generated while loading a key. This function does not work for VLRP (Very Low Power) and HSRUN (High Speed Run) modes Driver must be initialized (FCNFG[RAMRDY] == 0 and FCNFG[EEERDY] == 1)	

5. Appendix

5.1 API 사용시 유의사항

5.1.1 Autron HSM 프레임워크

5.1.1.1 수행시간

각 API 별 수행시간은 다음과 같으며 테스트 환경에 따라 수행 시간은 다르게 측정될 수 있으므로 단순 참고용으로 사용하도록 한다.

테스트 환경 : SPC58EC80, 현대차 표준플랫폼, 180 MHz(Core Clock)

함수	대상	최대 수행시간(us)	비고
Hsm_EnableSecureBoot	Secure Boot Size : 32K	1750	Runtime 중 Host Core 요청시
Hsm_GetSecureBootStatus	-	110	Sync 동작
Hsm_CustomlauExportData	HSM_DATA_CUSTOM_IAU_CERT	150	
	HSM_DATA_CUSTOM_IAU_UID	150	
	HSM_DATA_CUSTOM_IAU_PUBLIC_KEY	150	
	HSM_DATA_CUSTOM_IAU_DEFAULT_ISK	150	
Hsm_CustomlauPinningCheck	-	350	Sync 동작
Hsm_CustomlauGenSeedSc	Iteration 횟수 : 1024	50000	
Hsm_CustomlauKdf	-	300	

5.1.1.2 사용 예제

하기 예제 코드는 참조용으로 플랫폼 및 어플리케이션 설정, 구현 상황에 따라 단순 적용 불가함

공통 사항

Header File Include

```
/* Header file for Autron HSM driver */
#include "Hsm_AutDriver.h"
```

5.1.1.2.1 Hsm_EnableSecureBoot

Defines & Global Variables

```
/* Secure Boot 검증 Flash 시작 주소 */
#define APP_SECURE_BOOT_START_ADDRESS (0x00FC0000UL)
/* Secure Boot 검증 Flash 크기 */
#define APP_SECURE_BOOT_SIZE (0x8000UL)
```

예제

```
/* Secure Boot Enable 예제 */
Hsm_ErrType App_EnableSecureBoot(void)
{
    Hsm_ErrType LddRet;

    LddRet = Hsm_EnableSecureBoot(
        APP_SECURE_BOOT_START_ADDRESS, /* Secure Boot 시작 주소 */
        APP_SECURE_BOOT_SIZE           /* Secure Boot 크기 */
    );

    /* Secure Boot 활성화 실패시 원인 확인 */
    if (LddRet != HSM_NO_ERROR)
```

```
{
    /* Secure Boot 수행 결과 확인 */
    LddRet = Hsm_GetSecureBootStatus(
        &App_GddSecureBootStatus    /* Secure Boot 수행 결과 확인 변수 */
    );
}

return LddRet;
}
```

5.1.1.2.2 Hsm_GetSecureBootStatus

Global Variables

```
/* Secure Boot 수행 결과 확인 변수 */
HSM_SecureBootStatusType App_GddSecureBootStatus;
```

예제

```
/* Secure Boot 수행 결과 확인 */
LddRet = Hsm_GetSecureBootStatus(
    &App_GddSecureBootStatus
);
```

5.1.1.2.3 Hsm_CustomIauExportData

Defines & Global Variables

```
/* RootCA 길이 */
#define APP_Iau_ROOTCA_LENGTH          (217U)
/* IAU UID 길이 */
#define APP_Iau_UID_LENGTH              (13U)
/* IAU Public Key 길이 */
#define APP_Iau_PUBLIC_KEY_LENGTH      (64U)
/* IAU Default ISK 길이 */
#define APP_Iau_DEFAULT_ISK_LENGTH     (16U)
```

```
/* 추출된 IAU RootCA가 저장될 버퍼 */
uint8 App_GaalauCert[APP_Iau_ROOTCA_LENGTH];
/* 추출된 IAU UID가 저장될 버퍼 */
uint8 App_GaalauUID[APP_Iau_UID_LENGTH];
/* 추출된 IAU Public Key가 저장될 버퍼 */
uint8 App_GaalauPublicKey[APP_Iau_PUBLIC_KEY_LENGTH];
/* 추출된 IAU Default ISK가 저장될 버퍼 */
uint8 App_GaalauDefaultISK[APP_Iau_DEFAULT_ISK_LENGTH];
```

예제

```
/* RootCA 인증서 추출 예제 */
/* Sync 동작으로 수행 */
Hsm_ErrType App_IauExportCertCA(void)
{
    Hsm_ErrType LddHsmRet;

    LddHsmRet = Hsm_CustomIauExportData(
        FALSE,                /* Sync 동작 */
        NULL_PTR,             /* Sync 동작시 미사용 */
        HSM_DATA_CUSTOM_Iau_CERT, /* 추출하고자 하는 데이터 : RootCA 인증서 */
    );
}
```

```
APP_IAU_ROOTCA_LENGTH,      /* 추출하고자 하는 데이터 길이 : RootCA 인증서 길이 */
App_GaalauCert              /* 추출하고자 하는 데이터 저장 버퍼 */
);

if (LddHsmRet == HSM_NO_ERROR)
{
    /* RootCA 인증서 추출 성공 */
}
else
{
    /* RootCA 인증서 추출 실패 */
}

return LddHsmRet;
}

/* IAU UID 추출 예제 */
/* Sync 동작으로 수행 */
Hsm_ErrType App_lauExportUid(void)
{
    Hsm_ErrType LddHsmRet;

    LddHsmRet = Hsm_CustomlauExportData(
        FALSE,                /* Sync 동작 */
        NULL_PTR,            /* Sync 동작시 미사용 */
        HSM_DATA_CUSTOM_IAU_UID, /* 추출하고자 하는 데이터 : IAU UID */
        APP_IAU_UID_LENGTH,    /* 추출하고자 하는 데이터 길이 : IAU UID 길이 */
        App_GaalauUID         /* 추출하고자 하는 데이터 저장 버퍼 */
    );

    if (LddHsmRet == HSM_NO_ERROR)
    {
        /* IAU UID 추출 성공 */
    }
    else
    {
        /* IAU UID 추출 실패 */
    }

    return LddHsmRet;
}

/* IAU Public Key 추출 예제 */
/* Sync 동작으로 수행 */
Hsm_ErrType App_lauExportPublicKey(void)
{
    Hsm_ErrType LddHsmRet;

    LddHsmRet = Hsm_CustomlauExportData(
        FALSE,                /* Sync 동작 */
        NULL_PTR,            /* Sync 동작시 미사용 */
        HSM_DATA_CUSTOM_IAU_PUBLIC_KEY, /* 추출하고자 하는 데이터 : IAU Public Key */
        APP_IAU_PUBLIC_KEY_LENGTH,    /* 추출하고자 하는 데이터 길이 : IAU Public Key 길이 */
        App_GaalauPublicKey         /* 추출하고자 하는 데이터 저장 버퍼 */
    );
}
```

```

if (LddHsmRet == HSM_NO_ERROR)
{
    /* IAU Public Key 추출 성공 */
}
else
{
    /* IAU Public Key 추출 실패 */
}

return LddHsmRet;
}

/* IAU Default ISK 추출 예제 */
/* Sync 동작으로 수행 */
Hsm_ErrType App_lauExportDefaultISK(void)
{
    Hsm_ErrType LddHsmRet;

    LddHsmRet = Hsm_CustomlauExportData(
        FALSE,                /* Sync 동작 */
        NULL_PTR,             /* Sync 동작시 미사용 */
        HSM_DATA_CUSTOM_IAU_DEFAULT_ISK, /* 추출하고자 하는 데이터 : IAU DEFAULT ISK */
        APP_IAU_DEFAULT_ISK_LENGTH, /* 추출하고자 하는 데이터 길이 : IAU Default ISK 길이 */
        App_GaalauDefaultISK /* 추출하고자 하는 데이터 저장 버퍼 */
    );

    if (LddHsmRet == HSM_NO_ERROR)
    {
        /* IAU Default ISK 추출 성공 */
    }
    else
    {
        /* IAU Default ISK 추출 실패 */
    }

    return LddHsmRet;
}

```

5.1.1.2.4 Hsm_CustomlauPinningCheck

Global Variables

```

/* RootCA 인증서 Pinning Check 결과 저장 변수 */
boolean App_GbllauPinningCheck;

```

예제

```

/* RootCA 인증서 Pinning Check 예제 */
/* Sync 동작으로 수행 */
Hsm_ErrType App_lauPinningCheck(void)
{
    Hsm_ErrType LddHsmRet;

    App_GbllauPinningCheck = FALSE;
    LddHsmRet = Hsm_CustomlauPinningCheck(
        FALSE,                /* Sync 동작 */
        NULL_PTR,             /* Sync 동작시 미사용 */

```

```

    &App_GbllauPinningCheck    /* RootCA 인증서 Pinning Check 결과 저장 변수 */
);

if (LddHsmRet == HSM_NO_ERROR)
{
    /* IAU Pinning Check 호출 성공 */
    if (App_GbllauPinningCheck == TRUE)
    {
        /* IAU Pinning Check 성공 */
    }
    else
    {
        /* IAU Pinning Check 실패 */
    }
}
else
{
    /* IAU Pinning Check 호출 실패 */
}

return LddHsmRet;
}

```

5.1.1.2.5 Hsm_CustomlauGenSeedSc

Defines & Global Variables

```

/* 해시 반복 횟수 */
#define APP_IAU_GENSEEDSC_ITERATION_COUNT    (1024U)
/* 연결 스트링 길이 */
#define APP_IAU_GENSEEDSC_CONSTRING_LENGTH    (5U)
/* 생성된 SEED_SC 를 암호화/복호화 할 AES Key index */
#define APP_IAU_SEED_AES_KEY_INDEX    (1U)
/* 암호화된 SEED_SC 길이 */
#define APP_IAU_GENSEEDSC_ENC_SEED_SC_LENGTH    (32U)

/* 연결 스트링 버퍼 */
uint8 App_GaaConString[APP_IAU_GENSEEDSC_CONSTRING_LENGTH] =
{
    'S', 'I', 'O', 'T', '1'
};
/* 암호화된 SEED_SC가 저장될 버퍼 */
uint8 App_GaalauEncSeedSc[APP_IAU_GENSEEDSC_ENC_SEED_SC_LENGTH];
/* Async 동작 확인용 플래그 변수 */
boolean App_GblGenSeedScReq = FALSE;

```

예제

```

/* Async 요청에 대한 callback 처리 함수 */
/* Hsm으로부터 발생한 interrupt 처리 루틴 내의 호출이므로 수행시간이 긴 작업 절대 금지
   Ex) callback 루틴 내에서 암호화 알고리즘 요청 호출 */
void App_lauGenerateSeedSC_Cbk(Hsm_ErrType ddResult)
{
    /* Async 요청 확인 */
    if (App_GblGenSeedScReq == TRUE)
    {
        if (ddResult == HSM_NO_ERROR)

```

```

{
    /* SEED_SC 생성 성공 */
}
else
{
    /* SEED_SC 생성 실패 */
}
}

/* Async 처리를 위한 플래그 초기화 */
App_GblGenSeedScReq = FALSE;
}

/* Enc(SEED_SC) = Enc(SHA256n("ConString"+MS)) 생성 예제 */
/* Async 동작으로 수행 */
Hsm_ErrType App_lauGenerateSeedSc(void)
{
    Hsm_ErrType LddHsmRet;

    LddHsmRet = HSM_NO_ERROR;

    /* Async 이전 요청이 수행중인지 확인 */
    if (App_GblGenSeedScReq == FALSE)
    {
        LddHsmRet = Hsm_CustomlauGenSeedSc(
            TRUE, /* Async 동작 */
            App_lauGenerateSeedSc_Cbk, /* Async 요청 처리 완료시 호출될 callback 함수 포인터 */
            APP_IU_GENSEEDSC_ITERATION_COUNT, /* Hash Iteration 횟수 */
            App_GaaConString, /* 연결 스트링 버퍼 포인터 */
            APP_IU_GENSEEDSC_CONSTRING_LENGTH, /* 연결 스트링 길이 */
            (Hsm_DataIDType)APP_IU_SEED_AES_KEY_INDEX, /* 생성된 SEED_SC 를 암호화 할 AES Key index */
            App_GaalauEncSeedSc /* 암호화된 SEED_SC 가 저장될 버퍼 포인터 */
        );

        if (LddHsmRet == HSM_NO_ERROR)
        {
            /* SEED_SC 생성 요청 성공 */
            /* Async 처리를 위한 플래그 셋팅 */
            App_GblGenSeedScReq = TRUE;
        }
        else
        {
            /* SEED_SC 생성 요청 실패 */
            /* Async 처리를 위한 플래그 초기화 */
            App_GblGenSeedScReq = FALSE;
        }
    }
}

return LddHsmRet;
}

```

5.1.1.2.1 Hsm_CustomlauKdf

Defines & Global Variables

/* IAU 난수 길이 */

```
#define APP_IAU_RANDOM_NUMBER_LENGTH          (32U)
/* IAU Token ID 길이 */
#define APP_IAU_TOKEN_ID_LENGTH              (8U)
/* IAU AES Key 길이 */
#define APP_IAU_AES_KEY_LENGTH              (16U)

uint8 App_GaalauRandomNumber[APP_IAU_RANDOM_NUMBER_LENGTH] =
{
    /* 난수 */
};
uint8 App_GaalauTokenID[APP_IAU_TOKEN_ID_LENGTH] =
{
    /* Token ID */
};
uint8 App_GaalauAuthKey[APP_IAU_AES_KEY_LENGTH];
uint8 App_GaalauEncKey[APP_IAU_AES_KEY_LENGTH];
```

예제

```
/* IAU KDF 예제 */
/* Sync 동작으로 수행 */
Hsm_ErrType App_IauKdf(void)
{
    Hsm_ErrType LddHsmRet;

    LddHsmRet = Hsm_CustomIauKdf(
        FALSE,                                /* Sync 동작 */
        NULL_PTR,                            /* Sync 동작시 미사용 */
        App_GaalauRandomNumber,              /* 난수 버퍼 포인터 */
        App_GaalauTokenID,                   /* TokenID 버퍼 포인터 */
        App_GaalauEncSeedSc,                 /* 암호화된 Seed 버퍼 포인터 */
        (Hsm_DataIDType)APP_IAU_SEED_AES_KEY_INDEX, /* 암호화된 Seed를 복호화 하기 위한 AES Key ID */
        App_GaalauAuthKey,                   /* KDF 결과값 상위 16 byte 저장하기 위한 버퍼 포인터 */
        App_GaalauEncKey                     /* KDF 결과값 하위 16 byte 저장하기 위한 버퍼 포인터 */
    );

    if (LddHsmRet == HSM_NO_ERROR)
    {
        /* IAU KDF 성공 */
    }
    else
    {
        /* IAU KDF 실패 */
    }

    return LddHsmRet;
}
```

5.1.2 RH850 F1K 계열 ICUS

5.1.2.1 수행시간

CryptoLib 에서 제공하는 ICUS 용 API 는 sync 동작만을 지원한다. 따라서 API 를 사용하는 사용자측에서 blocking 동작으로 인한 OS_LIMIT 등의 문제가 발생하지 않도록 스케줄링 등의 노력을 기울여야 한다.

5.1.2.2 사용 가이드

CryptoLib 의 ICUS 드라이버를 직접 사용하고자 할 경우 Rte 레이어를 위한 인터페이스가 제공되지 않으므로, 사용자가 CDD 로 직접 구현해야 한다.

5.1.2.2.1 ICUS Key 등록

ICUS 는 ICUS 전용 Data Flash 에 원본 Key 값 추출이 불가능한 Key 를 주입할 수 있으며 Icus_EncEcb, Icus_DecEcb, Icus_EncCbc, Icus_DecCbc, Icus_GenerateMac, Icus_VerifyMac API 사용시에 사전에 주입한 Key 를 사용할 수 있으나, 해당 API 기능을 아래의 예제처럼 Autosar Csm 모듈의 표준 인터페이스를 통해 사용할 수 있으므로 Icus_xxx API 를 통한 직접 사용은 지양한다.

Ex) Icus_Rnd API 는 Csm 의 AutIcusRandomGenerate 설정을 통해 사용 가능

Container Details - CsmRandomGenerateConfig						
Index	Short Name	Callback Rand...	Init Configurat...	Primitive Name	Use Port	Use External T...
0	AutIcusRandomGenerate		NULL_PTR	AutIcusRandomGenerate	false	false

```

Csm_RandomGenerate(CSM_AUTICUSRANDOMGENERATE_INDEX,
    CryptoTest_GaaAutIcusRngGeneratedNum,
    CryptoTest_GaaAutIcusTrueRandomGeneratedLength);

```

Key 등록, 업데이트는 M1, M2, M3 메시지를 인자로 사용하며 M4, M5 는 해당 Key 의 등록, 업데이트를 확인하는데 사용된다. M1, M2, M3, M4, M5 는 SHE 사양을 참조한다.

M1, M2, M3 는 Icus_GenerateProtectedKey API 로 만들며 해당 인자로 Icus_LoadKey API 를 사용하여 Key 를 등록, 업데이트한다.

등록된 Key 는 Icus_Debug 를 통해 삭제하기 전까지 ICUS 전용 Data Flash 에 저장되어 있으므로, Key 등록을 위한 코드는 한 번만 적용되도록 한다.

Csm 을 통한 사전에 주입한 Key 사용 방법은 Csm 모듈 UM 의 8.4.4 Key 사용 방식을 참조하며 key index 값은 Icus_KeyIDType 의 enum 값을 사용한다.

Ex) Csm_SymKeyType 구조체를 통해 아래와 같이 KEY_1 키를 Csm 모듈에서 사용

```

Csm_SymKeyType Test_GddSymPreloadedKey = {
    1,
    {
        4 /* KEY_1 */
    }
};

```

Ex) Icus_EncEcb API 는 Csm 의 AutIcusAesEcbEncrypt 설정을 통해 사용 가능

Container Details - CsmSymBlockEncryptConfig						
Index	Short Name	Callback Sym ...	Init Configurat...	Primitive Name	Use Port	Use External T...
0	AutIcusAesEcbEncrypt		NULL_PTR	AutIcusAesEcbEncrypt	false	false

```

Std_ReturnType LddReturnValue;
uint8 Test_GaaAesEcbTextBuffer[16];
uint32 Test_GulCipherTextLength = 16;

LddReturnValue = Csm_SymBlockEncryptStart(CSM_AUTICUSAESECBENCRYPT_INDEX,
    &Test_GddSymPreloadedKey);

if (LddReturnValue == E_OK)
{
    LddReturnValue = Csm_SymBlockEncryptUpdate(CSM_AUTICUSAESECBENCRYPT_INDEX,
        Test_GaaAesEcbPlainText, 16, Test_GaaAesEcbTextBuffer,
        &Test_GulCipherTextLength);
}

if (LddReturnValue == E_OK)
{
    LddReturnValue = Csm_SymBlockEncryptFinish(CSM_AUTICUSAESECBENCRYPT_INDEX);
}

if (LddReturnValue == E_OK)
{
    /* AES ECB 암호화 성공 */
}

```

5.1.2.3 사용 예제

하기 예제 코드는 참조용으로 플랫폼 및 어플리케이션 설정, 구현 상황에 따라 단순 적용 불가함

Header File Include

```

/* Header file for ICUS driver */
#include "Icus_Driver.h"

```

Defines & Global Variables

```

/* MASTER_ECU_KEY raw key 값 */
const uint8 Test_GaalCusMasterEcuKey[ICUS_BLOCK_BYTE_LENGTH] = {
    0xAFU, 0x01U, 0x02U, 0x03U, 0x04U, 0x05U, 0x06U, 0x07U,
    0x08U, 0x09U, 0x0AU, 0x0BU, 0x0CU, 0x0DU, 0x0EU, 0x0FU
};

/* '0' key */
const uint8 Test_GaalCusZeroKey[ICUS_BLOCK_BYTE_LENGTH] = {
    0x00U, 0x00U, 0x00U, 0x00U, 0x00U, 0x00U, 0x00U, 0x00U,
    0x00U, 0x00U, 0x00U, 0x00U, 0x00U, 0x00U, 0x00U, 0x00U
};

/* Replay attack 방지용 counter */
uint32 Test_GulCusKeyCounter = 1;

/* M1, M2, M3, M4, M5 구조체 변수 */
Icus_ProtectedKey Test_GddIcusProtectedKey;

/* Key 등록/업데이트 검증 결과 */
boolean Test_GblIcusVerRes;

/* 수행 결과 */
Icus_ErrType Test_GddIcusRet;

```

MASTER_ECU_KEY 등록 예제

```

/* MASTER_ECU_KEY 에 대한 Icus_ProtectedKey (M1, M2, M3) 를 생성 */
Test_GddIcusRet = Icus_GenerateProtectedKey(
    MASTER_ECU_KEY,          /* MASTER_ECU_KEY 를 대상 */
    Test_GaalIcusMasterEcuKey, /* MASTER_ECU_KEY raw key 값 버퍼. 추후 다른 Key 등록/업데이트,
                               모든 Non-volatile memory Key 삭제에 사용하므로 매우 중요하며
                               반드시 외부 노출되지 않도록 보안을 유지해야 한다. */
    MASTER_ECU_KEY,          /* MASTER_ECU_KEY 는 MASTER_ECU_KEY 의 AuthKey 가 될 수 있다 */
    Test_GaalIcusZeroKey,     /* 최초 MASTER_ECU_KEY 등록시 '0' 으로 채워진 AuthKey 값을 사용한다. */
    Test_GullIcusKeyCounter,  /* 최초 등록시에는 counter 값을 1 이상의 값으로 하며,
                               업데이트시에는 기존의 counter 값보다 반드시 큰 값을 사용해야 한다. */
    ALL_ENABLED,             /* 모든 제약이 없는 Key flag 값. Icus_KeyFlagType 참조 */
    &Test_GddIcusProtectedKey /* 호출 성공시 Icus_ProtectedKey 구조체의 M1, M2, M3 멤버값이 생성된다. */
);

if (Test_GddIcusRet == ERC_NO_ERROR)
{
    /* Icus_GenerateProtectedKey 로 생성한 M1, M2, M3 를 인자로 MASTER_ECU_KEY 를 ICUS 내부에 등록 */
    Test_GddIcusRet = Icus_LoadKey(
        MASTER_ECU_KEY,          /* MASTER_ECU_KEY 를 대상 */
        &Test_GddIcusProtectedKey /* 호출 성공시 Icus_ProtectedKey 구조체의 M4, M5 멤버값이 생성된다. */
    );
    if (Test_GddIcusRet == ERC_NO_ERROR)
    {
        /* Icus_LoadKey 로 MASTER_ECU_KEY 가 정상적으로 등록되었는지 M4, M5 로 확인 */
        Test_GddIcusRet = Icus_VerifyProtectedKey(
            MASTER_ECU_KEY,          /* MASTER_ECU_KEY 를 대상 */
            Test_GaalIcusMasterEcuKey, /* Icus_GenerateProtectedKey 에 사용한 동일한 값 사용 */
            MASTER_ECU_KEY,          /* Icus_GenerateProtectedKey 에 사용한 동일한 값 사용 */
            Test_GullIcusKeyCounter,  /* Icus_GenerateProtectedKey 에 사용한 동일한 값 사용 */
            &Test_GddIcusProtectedKey, /* Icus_LoadKey 호출 후 생성된 M4, M5 전달 */
            &Test_GbllIcusVerRes      /* MASTER_ECU_KEY 등록/업데이트 검증 값 */
        );
        if (Test_GddIcusRet == ERC_NO_ERROR)
        {
            if (Test_GbllIcusVerRes == TRUE)
            {
                /* MASTER_ECU_KEY 등록 성공 */
            }
            else
            {
                /* MASTER_ECU_KEY 등록 실패 */
            }
        }
    }
}
}

```

KEY_1 등록 예제

```

/* KEY_1 등록 */
Test_GddIcusRet = Icus_GenerateProtectedKey(
    KEY_1,
    Test_GaalIcusAesKey1,
    MASTER_ECU_KEY,
    Test_GaalIcusMasterEcuKey,
    Test_GullIcusKeyCounter,
    ALL_ENABLED,

```

```

&Test_GddIcusProtectedKey
);
if (Test_GddIcusRet == ERC_NO_ERROR)
{
    Test_GddIcusRet = Icus_LoadKey(
        KEY_1,
        &Test_GddIcusProtectedKey
    );
    if (Test_GddIcusRet == ERC_NO_ERROR)
    {
        Test_GddIcusRet = Icus_VerifyProtectedKey(
            KEY_1,
            Test_GaalIcusAesKey1,
            MASTER_ECU_KEY,
            Test_GullIcusKeyCounter,
            &Test_GddIcusProtectedKey,
            &Test_GbllIcusVerRes);
        if (Test_GddIcusRet == ERC_NO_ERROR)
        {
            if (Test_GbllIcusVerRes == TRUE)
            {
                /* KEY_1 등록 성공 */
            }
            else
            {
                /* KEY_1 등록 실패 */
            }
        }
    }
}
}
}

```

모든 Non-volatile memory Key, RAM_KEY 삭제 예제

```

/* 모든 Non-volatile memory Key, RAM_KEY 삭제 */
Test_GddIcusRet = Icus_Debug(
    Test_GaalIcusMasterEcuKey          /* ICUS 사전에 등록된 MASTER_ECU_KEY raw key 값 버퍼 */
);
if (Test_GddIcusRet == ERC_NO_ERROR)
{
    /* Key 삭제 성공 */
}
else
{
    /* Key 삭제 실패 */
}
}

```

5.1.3 S32K 계열 CSEC

5.1.3.1 수행시간

CryptoLib 에서 제공하는 CSEC 용 API 는 sync 동작만을 지원한다. 따라서 API 를 사용하는 사용자측에서 blocking 동작으로 인한 OS_LIMIT 등의 문제가 발생하지 않도록 스케줄링 등의 노력을 기울여야 한다.

5.1.3.2 CSEC Enable, Disable

CSEC Enable 은 CSEC 설정 적용 바이너리의 최초 부트시 Csm_Init 수행과정에서 Data Flash 와 RAM 의 일부(CSEC 설정 참조) 가 CSEC 전용으로 변경된다. 이 때 대략 140 ms 정도의 수행시간이 걸리며 적용 성공시 FTFC.FCNFG 레지스터가

RAMRDY : Not ready, EEERDY : Ready 로 변경된다. 이 후 부팅시 이 과정은 생략된다.

과도한 수행시간이 발생하기 때문에 의도하지 않은 Wdg reset 이 발생할 수 있으므로 Csm_Init 이 Wdg_Init 이전에 수행되도록 해야한다.

FTFC (Flash Memory Module)

FSTAT	80	CCIF	Completed	RDCOLERR	No error	ACCERR	No error	FPVIOL	Not violated
FCNFG	01	MGSTAT0	No error	RDCOLLIE	Disabled	ERSAREQ	Not requested	ERSSUSP	Not suspended
		CCIE	Disabled						
		RAMRDY	Not ready	EEERDY	Ready				

CSEC Disable 은 Csec_Debug API 수행이 성공해야 하며 수행시간이 대략 175 ms 정도 걸린다. 해당 API 수행 성공 이후 다음 Reset 부터 CSEC Disable 이 적용되며 FTFC.FCNFG 레지스터가 RAMRDY : Ready, EEERDY : Not Ready 상태로 확인할 수 있다.

FTFC (Flash Memory Module)

FSTAT	80	CCIF	Completed	RDCOLERR	No error	ACCERR	No error	FPVIOL	Not violated
FCNFG	02	MGSTAT0	No error	RDCOLLIE	Disabled	ERSAREQ	Not requested	ERSSUSP	Not suspended
		CCIE	Disabled						
		RAMRDY	Ready	EEERDY	Not ready				

5.1.3.3 사용 가이드

CryptoLib 의 CSEC 드라이버를 직접 사용하고자 할 경우 Rte 레이어를 위한 인터페이스가 제공되지 않으므로, 사용자가 CDD 로 직접 구현해야 한다.